

People's Democratic Republic of Algeria
Ministry of Higher Education and Scientific Research
University M'Hamed BOUGARA – Boumerdes



Institute of Electrical and Electronic Engineering
Department of Electronics

Final Year Project Report Presented in Partial Fulfilment of the
Requirements for the Degree of

MASTER

In Electronic

Option: Computer Engineering

Entitled

**Image Compression and Decompression
using Deep Convolutional Autoencoder**

Presented by

- **Mr. Zerabib Abdellah**
- **Mr. Hammi Nadjem eddine**

Supervisor

- **Dr. Touzout Walid**

Registration Number:...../2024

Abstract

In the last decades, image compression has become a very important task since the Images contribute to a significant amount of internet traffic, demanding efficient solutions to reduce storage and transmission costs while maintaining image quality.

This thesis explores the use of Convolutional Autoencoders for image compression and decompression. Convolutional Autoencoders are employed to learn compact, meaningful representations of input images, which are then used to reconstruct the original images with minimal loss of quality. The model was tested using the Peak Signal-to-Noise Ratio (PSNR) and achieved good results, demonstrating the effectiveness of the compression.

Along with decompressing images, a network was trained to classify them bypassing the need for image reconstruction. The trained model compresses, classifies, and saves images efficiently, thus showing that compressed images can still be accurately categorized.

Dedication

Abdellah Zerabib.

I dedicate this work to my beloved parents. Your support, love, sacrifices and Duaa have made everything possible.

To my siblings, whose love and support have always brought me joy and inspiration.

Thank you for supporting me through every obstacle and celebrating each success.

To the rest of my family, of my family, whose love and encouragement have always uplifted me.

To my friends, you know who you are. Your friendship, laughter, and support have been invaluable.

To my project partner, Hammi Nadjem Eddine. Thank you for your dedication and hard work throughout this project. Your contributions have been invaluable, and I am grateful to have worked with you.

Finally, and especially, to my brother Mohamed. Your guidance, support, and unwavering belief in me have been beyond words. I wouldn't be anything without you. Thank you from the bottom of my heart for always being there for me, for your endless support, and for believing in me even when I doubted myself. Your influence has been profound and immeasurable.

Hammi Nadjem Eddine

This work is dedicated to my cherished family members: my affectionate parents, supportive brother, caring sister, and my remarkable friends. Each of you has played a profound role in shaping my journey and providing unwavering support throughout.

To my dear mother, the very reason for my existence, and to my venerable father, your unwavering love, guidance, and sacrifices have been the cornerstone of my success. Your support and encouragement have empowered me to overcome obstacles and pursue my aspirations. I am eternally grateful for your steadfast presence in my life, shaping me into the person I am today.

To my supportive brother and caring sister, you have been my pillars of strength and my closest confidants. Your encouragement, empathy, and unwavering support have spurred me to push my limits and achieve my goals. I cherish the bond we share and the memories we've created together.

To my exceptional circle of friends, and to my binôme, Abdellah Zerabib, your goodwill, dedication, and camaraderie have deeply influenced me. Thank you for standing by me through every challenge and triumph, and for believing in me. And to my friends, Anouar Tebbi, Founas Abd erraof, Khalil Bouchelaghem, and Azzeedine Amrani, you've been my constant source of inspiration, laughter, and unwavering faith. Your enduring support has been invaluable to me. Thank you.. Thank you.

Acknowledgements

First and foremost, we are deeply grateful to Allah for the strength, guidance, and blessings that have empowered us to complete this work. Alhamdulillah.

We are sincerely thankful to our supervisor, Dr. Touzout Walid, for his insightful guidance, valuable feedback. His support has been crucial in shaping this project.

We also want to express our gratitude to our co-supervisor, Bouchair Maria, whose assistance and expertise were instrumental in the development and completion of this work.

A special thank you to our friend Aymen Korichi for his invaluable assistance in assessing our compressed-decompressed images with his classification model.

Lastly, we are thankful to all our professors and colleagues for their assistance and to everyone who contributed, both directly and indirectly, to the success of this endeavor. Your help and encouragement have been greatly appreciated.

Table of Contents

Abstract	i
Dedication	ii
Acknowledgements	iv
List of Figures	x
List of Tables	xii
List of Abbreviations	xiii
General Introduction	xiv
1 Theoretical Background	1
1.1 Introduction	1
1.2 Image Compression	2
1.2.1 Importance and need of Compression	2
1.2.2 Image Compression Types	2
1.2.2.1 Conventional Methods	2
Lossless Image Compression:	2
Lossy Image Compression	2
1.2.2.2 Modern Techniques	3
1.3 AI-Based Image Compression	4
1.3.1 Overview of Artificial intelligence	4

TABLE OF CONTENTS

1.3.2	Evolution of AI in Image Compression	5
1.4	Machine Learning	6
	Supervised Learning	6
	Unsupervised Learning	6
	Reinforcement Learning	6
	Semi-Supervised Learning	6
1.5	Deep Learning	7
1.5.1	Artificial Neural Networks	7
1.5.2	Perceptron Artificial Neural Network	8
1.5.3	Multilayer Perceptron (MLP)	9
	1.5.3.1 FeedForward Artificial Neural Network	10
	1.5.3.2 FeedBackward Artificial Neural Network	10
1.5.4	Training Neural Networks	10
	1.5.4.1 Data preparation	11
	1.5.4.2 Data Splitting	12
	Training Set	12
	Validation Set	12
	Test Set	12
	1.5.4.3 Neural Network Architecture	12
	1.5.4.4 Optimization and Stochastic Gradient Descent	13
	1.5.4.5 Backpropagation	13
	1.5.4.6 Hyperparameter Tuning	13
	1.5.4.7 Loss functions	14
	1.5.4.8 Transfer Learning	15
1.5.5	Convolutional Neural Networks	16
1.5.6	Layers	16
1.5.7	Process of Convolution or Feature Extraction	17
	1.5.7.1 Convolutional layer components	18

TABLE OF CONTENTS

	Kernel	18
	Stride	18
	Padding	18
1.5.8	Activation functions	19
1.5.8.1	Rectified Linear Unit (ReLU)	19
1.5.8.2	Hyperbolic Tangent (tanh)	20
1.5.8.3	Leaky Rectified Linear Unit (Leaky ReLU)	21
1.5.8.4	Sigmoid	21
1.6	Advanced Neural Network Architectures for Image Compression . . .	22
1.6.1	Generative Adversarial Network	22
1.6.2	Autoencoder architecture	23
1.6.3	Types of Autoencoders	24
1.6.3.1	Undercomplete Autoencoder	24
1.6.3.2	Sparse Autoencoder	25
1.6.3.3	Contractive Autoencoder	25
1.6.3.4	Denoising Autoencoder	26
1.6.3.5	Convolutional Autoencoder	26
1.6.3.6	Variational Autoencoder	26
1.6.4	Applications of autoencoders	26
1.6.4.1	Dimensionality reduction	27
1.6.4.2	Image denoising	27
1.6.4.3	Image and Audio Compression	27
1.6.4.4	Generation of image and time series data	27
1.6.4.5	Recommender System	28
1.6.4.6	Anomaly detection	28
1.6.5	Limitations of Autoencoders	28
1.7	Metrics for Evaluating the Image Compression Quality	29
1.7.1	Mean Squared Error (MSE)	29

TABLE OF CONTENTS

1.7.2	Peak Signal-to-Noise Ratio PSNR	29
1.7.3	Structure Similarity Index Method (SSIM)	29
1.7.4	Compression Ratio	30
1.8	Evaluation metrics for Classification	30
	Accuracy	30
	F1	31
	Recall	31
	Precision	31
	Confusion Matrix	31
1.9	Quantization	32
	Linear Quantization:	32
	Range-Based Quantization:	32
1.10	Conclusion	32
2	Methodology	34
2.1	Introduction	34
2.2	Implementation Tools	34
2.2.1	Software and Platforms	34
	Python	34
	Jupyter Notebook:	35
	Google Colab	35
2.2.2	Libraries	35
	Pytorch	35
	torchvision	35
	Numpy	36
	Matplotlib	36
	Pillow	36
2.3	Data Collection	36
2.3.1	Overview of DIV2K Dataset	37

TABLE OF CONTENTS

2.4	Data Preparation	37
2.4.1	Image Loading and Preprocessing	37
2.4.2	Data Augmentation	37
2.4.3	Data Splitting	38
2.5	Custom Model Development	38
2.5.1	Experimental Network Designs	38
2.5.2	Best Performing network architecture	39
2.5.3	Training the Autoencoder	42
2.5.3.1	Fine-Tuning with Layer-Progressive Training	42
2.5.3.2	Model evaluation	43
2.6	Utilizing Compressed Data for Classification	43
2.6.1	Overview	43
2.6.2	Data Collection	44
2.6.3	Data Preparation	44
	Image Loading and Preprocessing	44
	Data augmentation	45
	Data splitting	45
2.6.4	Training a Classification Model	45
2.6.4.1	Model Architecture	45
2.6.4.2	Training Process	47
2.6.4.3	Evaluation of Classification and Reconstruction Per- formances	48
2.7	Conclusion	48
3	Results and Discussion	49
3.1	Introduction	49
3.2	Performance Evaluation of the Compression-Decompression Model . .	49
3.2.1	Prototype Autoencoder Network	49
3.2.2	Progressive Iterative Network	51

3.2.3	Optimized Autoencoder Model: Peak Performance Achieved . . .	52
3.2.4	Fine-Tuning Training Results	55
3.3	Performance Analysis of Classification with Compressed Images . . .	56
3.4	Comparision	59
3.5	Conclusion	60
General Conclusion		62
Bibliography		63

List of Figures

1.1	Subfields of Artificial Intelligence	5
1.2	Perceptron Neural Network	8
1.3	Multilayer Perceptron (MLP)	9
1.4	Feature Extraction	18
1.5	ReLU Activation Function	20
1.6	Tanh Activation Function	20
1.7	Leaky ReLU Activation Function	21
1.8	Sigmoid Activation Function	22
1.9	Standard architecture of the GAN	23
1.10	Architecture of an Autoencoder	23
1.11	Architecture of a Sparse Autoencoder	25
2.1	Architecture of the Autoencoder	41
2.2	Architecture Diagram of the Classification Neural Network	46
3.1	MSE loss plotting for the scratch network.	50

3.2	PSNR plotting for the scratch network.	50
3.3	Original Image	50
3.4	Original image cropped 256x256	51
3.5	Compressed-decompressed from the scratch net	51
3.6	Visual Assessment of Scratch Network Performance	51
3.7	PSNR and MSE loss plotting for the refined network	52
3.8	Original image	52
3.9	Compressed-decompressed	52
3.10	Original image vs compressed-decompressed.	52
3.11	MSE loss for the best network.	53
3.12	PSNR plotting for the optimized network	54
3.13	Original image	54
3.14	Compressed-decompressed	54
3.15	Original image vs compressed-decompressed.	54
3.16	Evaluation of the classification with original data	55
3.17	Evaluation of the classification with reconstructed data	55
3.18	PSNR of the train and validation	56
3.19	PSNR of the train and validation	56
3.20	MSE Loss	57
3.21	Cross entropy Loss	57
3.22	The loss plot for both reconstruction and Classification.	57
3.23	PSNR of the train and validation	57
3.24	Accuracy of train and validation	57
3.25	PSNR and accuracy plots for Reconstruction and classification. . . .	57
3.26	Original image for classification and reconstruction	58
3.27	reconstructed image	58
3.28	Original image vs compressed-decompressed.	58
3.29	Confusion Matrix	59

List of Tables

1.1 Confusion Matrix 32

3.1 Classification report with precision, recall, and F1-score for each class. 58

3.2 Comparison of PSNR and Compression Ratio between different image
compression methods on the Kodak dataset. 60

List of Abbreviations

AI:	Artificial Intelligence
ANN:	Artificial Neural Network
CNN:	Convolutional Neural Network
SSIM:	Structural Similarity Index Measure
PSNR:	Peak Signal-to-Noise Ratio
VAE:	Variational Autoencoder
GAN:	Generative Adversarial Network
JPEG:	Joint Photographic Experts Group
WebP:	Web Picture Format
DIV2K:	Diverse 2K
GPU:	Graphics Processing Unit

Genenral Introduction

Since the emergence of the Internet in the late 20th century, the world has witnessed an exponential increase in the generation, transmission, and utilization of data. With this large increase in the volume of information, it becomes necessary to find effective ways to manage and store it. Pictures, especially, make up a big part of internet traffic, highlighting the importance of efficient storage methods. One effective solution for managing large volumes of image data is image compression.

Image compression is a process applied to a graphics file to minimize its size without degrading image quality beyond an acceptable level. Over the years, many methods have been proposed. The most used image compression techniques are traditional methods like PNG (Portable Network Graphics) and JPEG (Joint Photographic Experts Group).

These techniques perform well in reducing file size while maintaining image quality, making them ideal for everyday use in web applications and digital photography. However, in image processing, using the compressed forms of PNG and JPEG images to train CNN models (without decompression) can be very challenging. The aim of this project is to develop an Autoencoder model for compressing diverse image types and achieving high compression ratios while maintaining good PSNR values, and showing that it is efficient to use compressed data to train CNN models.

This report offers comprehensive discussion of our work, It is organized into several chapters, each covering different aspects of the project. Chapter 1 introduces fundamental image compression and deep learnnig concepts, while Chapter 2 fo-

cuses on the implementation and evaluation of our compression-decompression and end-to-end compression-classification models. This chapter also illustrates the tools, techniques, used for model optimization. Chapter 3 presents the obtained results, compares them with existing approaches, and provides a thorough discussion, with a focus on the findings.

Chapter 1

Theoretical Background

1.1 Introduction

Image compression is essential in modern technology for optimizing bandwidth and storage. This chapter explores a variety of compression approaches, including AI-based techniques, which have revolutionized the field by enhancing efficiency and quality. Image compression plays a crucial role in telecommunications, multimedia, and medical imaging, enabling faster transmission and efficient storage. A comprehensive review of both conventional and modern compression methods is provided, with a focus on the evolution of AI in this domain. Key concepts in machine learning and deep learning, particularly convolutional neural networks (CNNs) and autoencoders, are introduced and discussed in the context of image compression. Additionally, the chapter examines important evaluation metrics such as Peak Signal-to-Noise Ratio (PSNR) and Structural Similarity Index (SSIM). Overall, the chapter offers a thorough overview of the theoretical foundations and advancements in image compression.

1.2 Image Compression

Image compression is a process applied to a graphics file to minimize its size in bytes without degrading image quality below an acceptable threshold. By reducing the file size, more images can be stored in a given amount of disk or memory space. The image also requires less bandwidth when being transmitted over the internet or downloaded from a webpage, reducing network congestion and speeding up content delivery[44].

1.2.1 Importance and need of Compression

The demand for efficient storage and transmission of digital images has increased with the development of high-resolution images and videos. As file sizes grow, storing and transmitting these files without compression would be impractical, consuming excessive storage space and bandwidth. Image compression addresses this challenge by reducing data size while preserving acceptable quality levels. This reduction not only conserves storage space but also accelerates transmission, This reduction helps apps like applications like web browsing, image sharing, and video streaming. Therefore, optimizing overall speed and user experience in digital media consumption requires effective image compression. [22].

1.2.2 Image Compression Types

1.2.2.1 Conventional Methods

- **Lossless Image Compression:** Lossless compression techniques encode all the information from the original image file, ensuring that when the image is decompressed, it is identical to the original. Formats such as PNG and GIF utilize lossless compression, making them suitable when it is crucial to preserve the exact original image data. The choice of compression format depends on the specific requirements of the image being compressed[49].

- **Lossy Image Compression:** Lossy compression techniques, as the name implies, result in the loss of some information. While the compressed image remains similar to the original, it is not an exact replica due to the data loss that occurs during compression. Techniques like JPEG are commonly used for lossy compression, as they are well-suited for images where perfect accuracy is not essential. These techniques employ algorithms that approximate the original image, necessitating quality measurements to evaluate the compression's impact. Lossy compression typically achieves a higher compression ratio compared to lossless compression.[45].

1.2.2.2 Modern Techniques

- **Hybrid Methods:** These techniques combine elements of both lossless and lossy compression to optimize for both quality and compression ratio. An example of a hybrid method is the WebP format developed by Google[25], which supports both lossy and lossless compression. In lossy mode, WebP uses predictive coding to encode an image, significantly reducing file size while maintaining visual quality. In lossless mode, it employs techniques like dedicated entropy coding and color indexing to compress image data without any loss of information
- **Advanced Algorithms:** Methods such as wavelet transforms provide more sophisticated ways to compress images, offering better quality at higher compression ratios. Wavelet-based compression methods, as implemented in the JPEG 2000 standard, decompose an image into wavelets representing image details at multiple scales. This allows for more efficient coding of image features, leading to higher compression ratios and improved image quality, especially at low bit rates[15].

1.3 AI-Based Image Compression

1.3.1 Overview of Artificial intelligence

AI is one of the newest fields in science and engineering. Work started in earnest soon after World War II, and the name itself was coined in 1956. Along with molecular biology, AI is regularly cited as the “field I would most like to be in” by scientists in other disciplines. A student in physics might reasonably feel that all the good ideas have already been taken by Galileo, Newton, Einstein, and the rest. AI, on the other hand, still has openings for several full-time Einsteins and Edisons. AI currently encompasses a huge variety of subfields, ranging from the general (learning and perception) to the specific, such as playing chess, proving mathematical theorems, writing poetry, driving a car on a crowded street, and diagnosing diseases. [39]AI is relevant to any intellectual task; it is truly a universal field. AI is an umbrella term covering a variety of interrelated, but distinct, subfields as shown in figure 1.1. Some of the most common fields you will encounter within the broader field of artificial intelligence include[41]:

- **Machine learning (ML):** Machine learning is a subset of AI in which algorithms are trained on data sets to become machine learning models capable of performing specific tasks.
- **Deep Learning:** Deep learning is a subset of ML, in which artificial neural networks (AANs) that mimic the human brain are used to perform more complex reasoning tasks without human intervention.
- **Natural Language Processing:** A subset of computer science, AI, linguistics, and ML, natural language processing focuses on creating software capable of interpreting human communications.

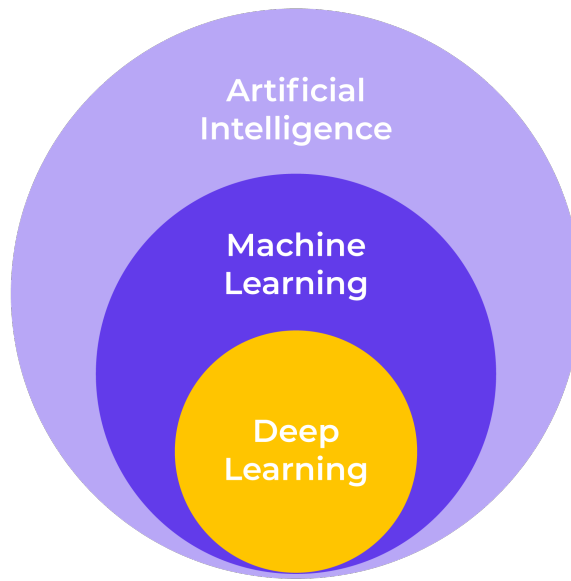


Figure 1.1: Subfields of Artificial Intelligence

1.3.2 Evolution of AI in Image Compression

The evolution of AI in image compression can be traced back to the early development of neural networks. Initially, traditional methods dominated, but the advent of deep learning brought significant advancements. AI-based image compression began gaining traction in the 2000s with the introduction of deep autoencoders and convolutional neural networks (CNNs), which showed promising results in learning efficient image representations [32].

Deep autoencoders, in particular, have played an important role by learning compact latent representations of images, enabling effective compression through reduced redundancy while preserving essential features [28]. Current trends focus on end-to-end optimization and advanced autoencoder architectures, demonstrating superior performance in maintaining image quality and achieving high compression ratios [7].

1.4 Machine Learning

Machine learning is a subject that studies how to use computers to simulate human learning activities, and to study self-improvement methods of computers that obtain new knowledge and new skills, identify existing knowledge, and continuously improve the performance and achievement. Compared to human learning, machine learning is faster, allows for easier accumulation of knowledge, and facilitates the spread of learning outcomes more efficiently[46]. So, any progress of human in the field of machine learning, will enhance the capability of computers, thus have an impact on human society.

Today, machine learning is the primary means through which most people interact with AI. Machine learning (ML) include four main types of knowledge acquisition:

- **Supervised Learning:** Supervised learning is a type of machine learning where an algorithm is trained on labeled data, meaning the input data is paired with the correct output. The algorithm learns to map inputs to outputs by finding patterns in the training data, enabling it to make predictions on new, unseen data. This approach is commonly used for tasks like classification and regression[27].
- **Unsupervised Learning:** Unsupervised learning is a type of machine learning where an algorithm is trained on unlabeled data, meaning the input data has no corresponding output labels. The algorithm tries to find hidden patterns or intrinsic structures in the data, often used for clustering and association tasks.[27].
- **Reinforcement Learning:** Reinforcement learning is a type of machine learning where an agent learns to make decisions by performing actions in an environment to maximize cumulative reward. The agent receives feedback through rewards or penalties and uses this to improve its strategy over time[42].

- **Semi-Supervised Learning:** Semi-supervised learning is a type of machine learning that uses a combination of labeled and unlabeled data to improve learning accuracy. It leverages the small amount of labeled data to guide the learning process, while utilizing the larger amount of unlabeled data to enhance model performance and generalization[9].

1.5 Deep Learning

Deep learning is a subset of machine learning that focuses on creating artificial neural networks with multiple layers, mimicking the human brain's structure and function. It has gained popularity due to its ability to learn hierarchical representations from raw data, often outperforming state-of-the-art methods in various fields. Deep learning is based on interconnected neurons grouped in layers, which use mathematical modifications to process input signals. This structure enables models to learn complex features from large datasets, leading to advances in computer vision, natural language processing, and predictive maintenance, and other areas[32].

Training deep neural networks requires substantial data and computational resources. However, advancements in cloud computing and the development of specialized hardware like GPUs have simplified the training process. These technologies have lowered the barriers for researchers and practitioners, making it easier to adopt and experiment with deep learning models.[24].

1.5.1 Artificial Neural Networks

An Artificial Neural Network (ANN) is best understood as a weighted directed graph with interconnected nodes mimicking human brain neurons[13]. The first layer receives raw input, while each subsequent layer processes the output, similar to neural signal processing in the brain. The final layer produces the system's output. ANNs are mathematical models that enhance data analysis technologies

and drive advancements in AI, machine learning, and deep learning.

1.5.2 Perceptron Artificial Neural Network

The perceptron is a fundamental form of artificial neural network used for binary prediction tasks[13]. It assumes linear separability in input data and assigns weights to each input value. It computes their weighted sum, incorporates a bias term, and applies an activation function to generate an output indicating the network's final prediction.. Illustrated in Figure 1.2, the perceptron's operation can be succinctly expressed through the following equation:

$$y = f \left(\sum_{i=1}^n w_i x_i + b \right) \quad (1.1)$$

where:

- y is the output,
- f is the activation function (e.g., step function),
- w_i are the weights,
- x_i are the input values,
- b is the bias term,
- n is the number of input values.

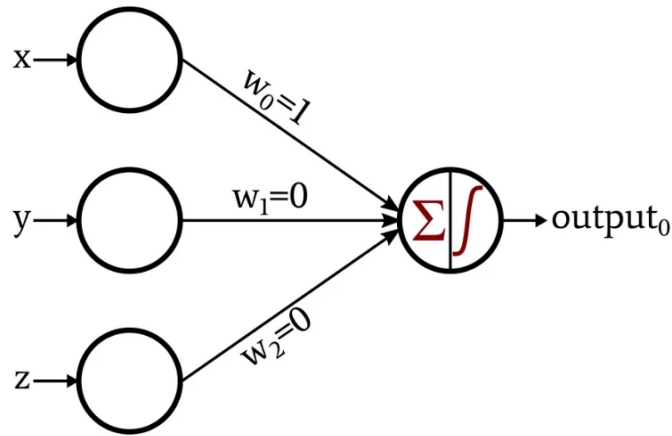


Figure 1.2: Perceptron Neural Network

1.5.3 Multilayer Perceptron (MLP)

A fully connected multi-layer neural network, known as a Multilayer Perceptron (MLP), as shown in the figure 1.3. This type of neural network consists of multiple layers of artificial neurons or nodes, which makes it suitable for tackling more complex classification and regression tasks. Examples include Convolutional Neural Networks and Recurrent Neural Networks. The standard model is a three-layer, fully connected backpropagation network. It comprises an input layer of neurons, which pass data to a hidden layer, and finally to an output layer[13]. MLPs can be categorized into two topologies: FeedForward and Feedback.

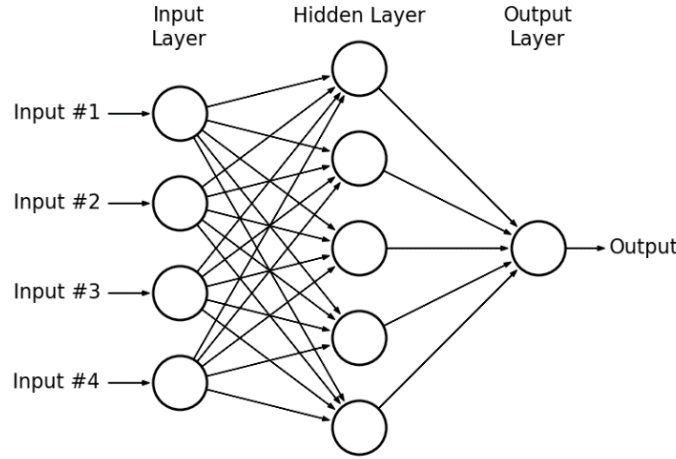


Figure 1.3: Multilayer Perceptron (MLP)

The output y of an MLP can be mathematically expressed by the following equation:

$$y = f \left(\sum_{j=1}^m w_{ij} f \left(\sum_{i=1}^n w_i x_i + b_i \right) + b_j \right) \quad (1.2)$$

where:

- y is the output,
- f is the activation function,
- w_{ij} are the weights from the hidden layer to the output layer,
- w_i are the weights from the input layer to the hidden layer,

- x_i are the input values,
- b_i and b_j are the bias terms,
- n is the number of input values,
- m is the number of neurons in the hidden layer.

1.5.3.1 FeedForward Artificial Neural Network

In this ANN, the information flow is unidirectional. The information travels only in one direction; forward; without making any feedback loops.[13] It first goes through the input nodes, then through the hidden nodes (in case there are any), and in the end, it goes through the output nodes.

1.5.3.2 FeedBackward Artificial Neural Network

In this case, the output of a neuron or a group of neurons is fed back into the network as input to the same or previous layers. This creates a recursive process where the network's output depends not only on the current input but also on previous outputs.[13] Feedback loops can occur in various types of neural network architectures, but they are particularly common in recurrent neural networks (RNNs).

1.5.4 Training Neural Networks

Training neural networks involves optimizing a mathematical model to make accurate predictions or classifications based on input data. This process involves preparing a dataset, choosing a suitable neural network architecture, and training the network using an optimization algorithm like gradient descent[32]. The model's parameters are adjusted iteratively based on the gradients of the loss function with respect to these parameters, using a method known as backpropagation. This technique calculates how each parameter contributes to the error, allowing the network to update its weights accordingly to improve its predictions. Hyperparameters, such as learning rate and batch size, may be fine-tuned to improve training performance.

Additionally, techniques like regularization and early stopping are employed to prevent overfitting and ensure generalization to unseen data [24]. Finally, the trained model is evaluated on the validation set to assess its performance and may be further refined before deployment. The main concepts and techniques required for training a neural network are presented in the following:

1.5.4.1 Data preparation

Data preparation is a crucial part in machine learning and deep learning, serving as the foundation for model training and evaluation. It involves a series of preprocessing steps aimed at transforming raw data into a format that is useful for analysis and modeling [30]. This process includes various techniques such as:

- **Data Preprocessing:** Data preprocessing involves cleaning and transforming raw data to address common challenges such as missing values, outliers, and inconsistencies. Techniques like normalization, standardization, and feature scaling are commonly used to make sure that the data is properly scaled and centered, facilitating convergence during training and improving the performance of machine learning algorithms [20].
- **Data Augmentation** Data augmentation is another essential component of data preparation, particularly in scenarios where the training dataset is limited or unbalanced. Data augmentation techniques involve generating synthetic data samples by applying transformations such as rotation, flipping, cropping, and color jittering to existing data. By augmenting the dataset with diverse variations of the original samples, data augmentation helps improve the generalization and robustness of deep learning models, enabling them to perform well on unseen data [40].

1.5.4.2 Data Splitting

For most machine learning tasks, Data Splitting involves creating three primary subsets: training set, validation set (optional), and test set. In essence, data splitting refers to dividing an available dataset into distinct groups based on specific criteria or proportions.

- **Training Set** The training set is used to train the model's parameters and is typically the largest subset of the data. The model learns from the patterns in this data to make predictions or classifications.
- **Validation Set:** The validation set is used to tune hyperparameters and monitor the model's performance during training. It helps identify potential issues such as overfitting and guides the selection of the final model.
- **Test Set:** The test set is used to evaluate the final model's performance on unseen data. It provides an unbiased estimate of the model's generalization ability and helps assess its readiness for deployment in real-world scenarios.

1.5.4.3 Neural Network Architecture

Neural network architecture is the foundation of a machine learning model, describing how information flows and processes inputs to produce outputs. It includes components like neuron arrangement, layers, and connections. Designing neural network architectures depends on the task complexity and input data nature [24]. For example, convolutional neural networks (CNNs) are well-suited for image recognition tasks due to their ability to capture spatial relationships, while recurrent neural networks (RNNs) do better in sequential data processing tasks such as natural language processing. The choice of architecture involves balancing model complexity and computational efficiency, often influenced by experimentation and domain knowledge.

1.5.4.4 Optimization and Stochastic Gradient Descent

Optimization in neural networks involves refining model parameters to minimize a specified loss function, aiming for optimal performance[24]. At the core of optimization lies gradient descent, a fundamental algorithm used to update parameters iteratively, calculating the gradient of the loss function for each parameter. Parameters are adjusted in the opposite direction of this gradient, progressively approaching a local minimum. Variations of gradient descent, such as stochastic gradient descent (SGD), Adam, and RMSProp, improve convergence speed and stability. SGD updates parameters using a subset of training samples, Adam combines adaptive learning rates and momentum, and RMSProp adapts learning rates based on historical gradients[38]. These optimization techniques are widely used in training neural networks across various domains, offering flexibility and efficiency in parameter optimization.

1.5.4.5 Backpropagation

Backpropagation is a crucial algorithm for training neural networks by computing the gradient of the loss function with respect to each parameter in the network. It propagates the error backward from the output layer to the input layer, applying the chain rule of calculus to compute the gradient at each layer. During the forward pass, input data is passed through the network, and predictions are made. The backward pass calculates the loss change for small changes in output predictions, adjusting the parameters of earlier layers to minimize overall loss. This gradient is then propagated backward through the network to update the parameters using gradient descent [24].

1.5.4.6 Hyperparameter Tuning

Hyperparameter tuning is the process of choosing the most effective hyperparameters for a machine learning model to optimize its performance on a specific

task. These hyperparameters, which are not learned during training but set before it, include learning rate, batch size, number of hidden layers, and activation functions. the tuning of this hyperparameters is typically performed using techniques like grid search, random search, or advanced optimization algorithms like Bayesian optimization. The goal is to find the hyperparameters that yield the best performance on a validation dataset[24]. Effective hyperparameter tuning can significantly improve the performance of machine learning models and is a crucial step in the model development process [8].

1.5.4.7 Loss functions

A loss function, also known as a cost function, is essential in artificial intelligence and machine learning. It is a mathematical function that quantifies the difference between the predicted and actual values in a dataset [24]. The purpose of a loss function is to evaluate the model's performance by measuring the accuracy of its predictions. Essentially, it calculates the error or loss incurred by the model, facilitating the optimization process to improve model performance [35]. The most commonly used loss functions include:

1. **Mean-Squared Error (MSE):** Mean Squared Error is a widely used loss function for regression tasks. It calculates the average squared difference between the predicted and actual values. The equation of the loss is:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (1.3)$$

where y_i represents the actual value, $\hat{y}_i$ represents the predicted value, and n is the number of samples. MSE penalizes larger errors more significantly than smaller ones, making it suitable for tasks requiring precise numerical predictions [24].

2. **Cross-Entropy Loss:** Cross-Entropy Loss is commonly used in classification

tasks, especially with categorical data. It measures the dissimilarity between the predicted probability distribution and the actual distribution of class labels. The equation of the loss is:

$$\text{Cross-Entropy Loss} = -\frac{1}{n} \sum_{i=1}^n \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c}) \quad (1.4)$$

where $y_{i,c}$ is a binary indicator (0 or 1) if class label c is the correct classification for observation i , $\hat{y}_{i,c}$ is the predicted probability that observation i belongs to class c , and C is the number of classes. Cross-Entropy Loss increases as the predicted probability diverges from the true label, pushing the model to produce more accurate probability distributions over classes [35].

3. **Mean-Absolute Error (MAE):** Mean Absolute Error is another loss function frequently used for regression tasks. Unlike MSE, which calculates squared differences, MAE computes the average absolute difference between the predicted and actual values. The equation of the loss is:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (1.5)$$

where y_i represents the actual value, $\hat{y}_i$ represents the predicted value, and n is the number of samples. MAE is less sensitive to outliers compared to MSE, making it suitable for datasets with significant variations or noise [24].

1.5.4.8 Transfer Learning

Transfer learning is a machine learning technique for training neural networks that leverages pre-trained models developed for one task as the starting point for a new, related task. This approach significantly reduces the amount of data and computational resources required, as the pre-trained model's parameters serve as a strong foundation. By fine-tuning these models, transfer learning enhances

performance[6], especially in scenarios with limited data, reduces training time, and mitigates overfitting. This technique is widely applied in fields such as natural language processing, computer vision, and speech recognition. Throughout this project, transfer learning techniques were utilized to optimize training processes, demonstrating notable improvements in accuracy and efficiency.

1.5.5 Convolutional Neural Networks

Convolutional neural networks (CNNs) are a specialized type of neural network designed for processing data with a known, grid-like topology, such as images. CNNs have demonstrated remarkable success in various practical applications, particularly in computer vision tasks. The term "convolutional neural network" reflects the use of convolution as a fundamental operation within the network architecture. Convolution is a specialized form of linear operation that involves combining input data with learnable filters to produce feature maps. CNNs leverage convolutional layers, where convolution is applied to input data, enabling the network to extract hierarchical features automatically. This distinctive feature makes CNNs highly effective for tasks like image processing, image classification, object detection, and image segmentation[24].

1.5.6 Layers

While the convolutional neural network (CNN) presents a complex architecture, it primarily consists of various types of layers, each serving a specific purpose[11]:

1. **Convolutional layers:** Convolutional layers apply convolution operations to the input, extracting high-level features by learning spatial hierarchies in the data. These layers are the core building blocks of a CNN, enabling the network to detect patterns such as edges, textures, and shapes in the input data[48].
2. **Pooling layers:** Pooling layers reduce the spatial dimensions of the feature

maps and help in down-sampling. This operation not only reduces the computational load but also helps in making the detection of features invariant to scale and orientation changes.

3. **Dropout layers:** Dropout layers prevent overfitting by randomly setting a fraction of the input units to zero during training. This technique forces the network to learn more robust features that are not reliant on the presence of particular neurons, thus improving generalization.
4. **Fully connected layers:** Fully connected layers integrate the features learned by previous layers to make final predictions. These layers are typically used at the end of the network to compile the results of convolutional and pooling operations into a final classification or regression output.
5. **Batch normalization layers:** Batch normalization layers play a crucial role in improving the training speed and stability of the network by normalizing the inputs of each layer, ensuring consistent distributions. This helps in reducing the internal covariate shift and allows for the use of higher learning rates.

1.5.7 Process of Convolution or Feature Extraction

Convolution is a key operation in convolutional neural networks (CNNs) used for feature extraction. It involves applying a small filter, known as a kernel or a convolutional filter, to the input data. The kernel is a small matrix of weights that slides across the input data as shown in Figure 1.4, computing the dot product between its values and the values of the input data within a local region. This process results in the generation of a feature map, which highlights the presence of certain patterns or features in the input data. By sliding the kernel across the entire input data, the CNN can extract various features at different spatial locations[48]. Additionally, convolutional layers often employ multiple kernels, each responsible for detecting different features. This allows the CNN to learn and extract complex hi-

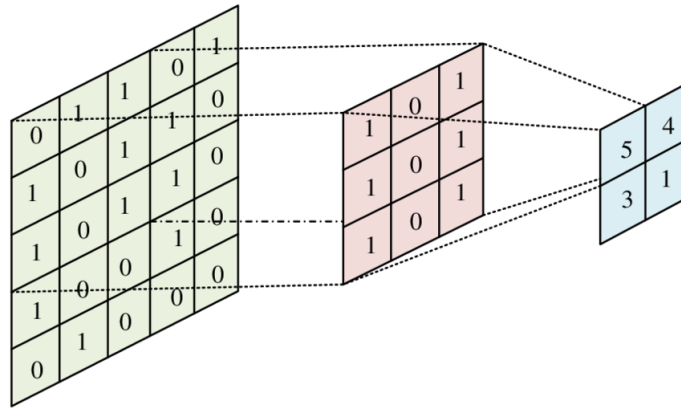


Figure 1.4: Feature Extraction

erarchical representations of features from the input data, enabling effective pattern recognition and classification.

1.5.7.1 Convolutional layer components

Understanding the fundamental components of convolutional neural networks (CNNs) is essential for grasping how these networks operate and extract features from input data. Three key components that significantly influence the convolutional process in CNNs are:

- **Kernel:** A kernel is a small matrix of weights used in CNNs to detect specific patterns or features in input data. It's also known as a filter or convolutional filter. The kernel's size and weights are learned during training, affecting the network's ability to extract relevant features from the input. By sliding the kernel across the data and computing the dot product with local regions, CNNs can generate feature maps highlighting these patterns.[48].
- **Stride:** Stride is a parameter in convolutional neural networks (CNNs) that controls the step size or distance the kernel moves across input data during convolution. A larger stride results in fewer computations and smaller output feature maps, while a smaller stride leads to more computations and larger output feature maps[48]. Adjusting stride affects the network's ability to capture spatial relationships and patterns..

- **Padding:** Padding in convolutional neural networks (CNNs) is essential when the kernel extends beyond the activation map borders. It maintains data integrity at the edges, improving network performance and preserving the input's spatial size, which supports the development of deeper networks. Zero-padding, the most common technique, involves adding zeros symmetrically around the input edges[48].

1.5.8 Activation functions

Activation functions are important components within neural networks, crucial for introducing non-linearity and enabling these models to approximate complex relationships within data. Essentially, an activation function operates on the weighted sum of inputs to a neuron, determining whether it should be activated or not. This activation is then propagated forward through the network. By integrating non-linear transformations, activation functions allow neural networks to learn and represent intricate patterns in data, contributing to the model's expressive power and capacity to generalize [24]. Various activation functions serve different purposes, each with distinct characteristics suited for specific tasks.

1.5.8.1 Rectified Linear Unit (ReLU)

ReLU is a commonly used activation function in neural networks. It sets all negative input values to zero and leaves positive values unchanged. Mathematically, it is defined as:

$$f(x) = \max(0, x) \tag{1.6}$$

ReLU is popular because it's simple and computationally efficient. However, it can lead to the "dying ReLU" problem where some neurons become inactive during training [4]. The ReLU activation function is shown in Figure 1.5.

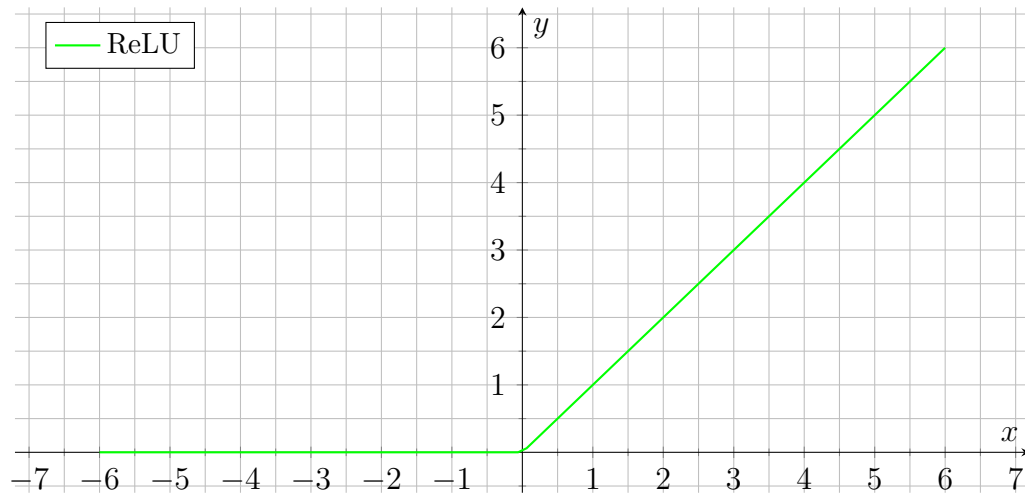


Figure 1.5: ReLU Activation Function

1.5.8.2 Hyperbolic Tangent (tanh)

The tanh function squashes input values to a range between -1 and 1. It's similar to the sigmoid function but centered around zero. Mathematically, it is defined as:

$$f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (1.7)$$

Tanh is useful in architectures where zero-centered data is preferred. However, it may still suffer from saturation for extreme inputs [32]. The tanh activation function is shown in Figure 1.6.

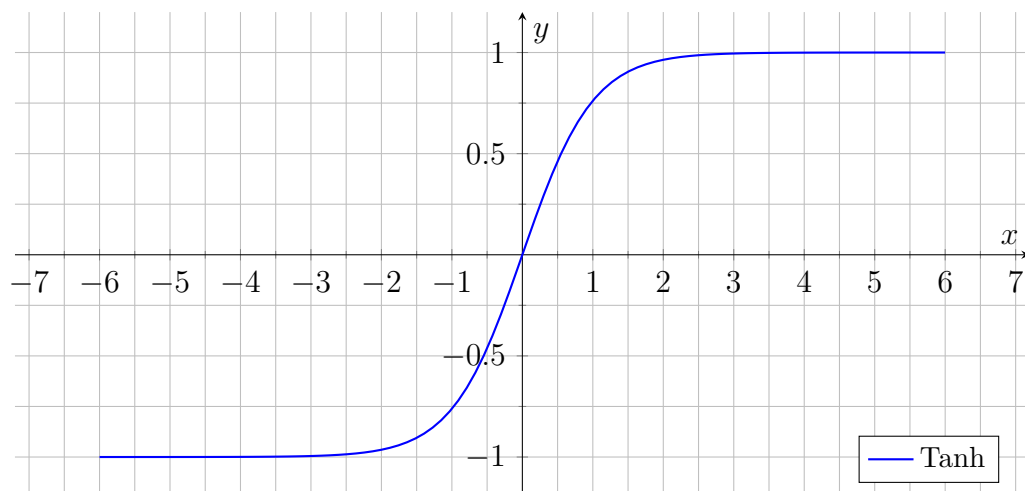


Figure 1.6: Tanh Activation Function

1.5.8.3 Leaky Rectified Linear Unit (Leaky ReLU)

Leaky ReLU is a modification of ReLU that allows a small, non-zero gradient for negative inputs. It prevents neurons from becoming dormant during training, addressing the "dying ReLU" problem while retaining the efficiency of ReLU. Mathematically, it is defined as:

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases} \quad (1.8)$$

where α is a small constant. The Leaky ReLU activation function is shown in Figure 1.7.

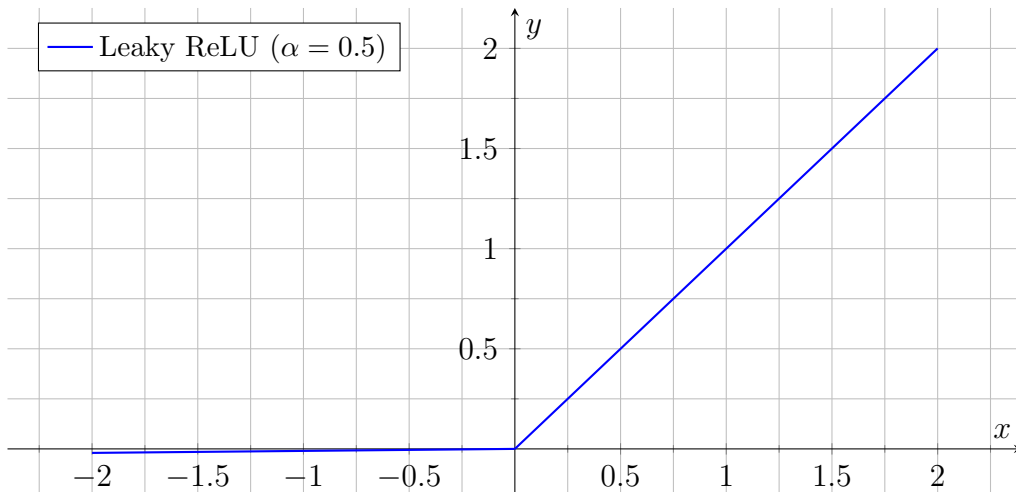


Figure 1.7: Leaky ReLU Activation Function

1.5.8.4 Sigmoid

The sigmoid function maps input values to a range between 0 and 1, useful for binary classification tasks. Mathematically, it is defined as:

$$f(x) = \sigma(x) = \frac{1}{1 + e^{-x}} \quad (1.9)$$

Despite its historical significance, sigmoid has limitations such as vanishing gradients and non-zero-centered output, making it less common in modern architectures [23].

The sigmoid activation function is shown in Figure 1.8.

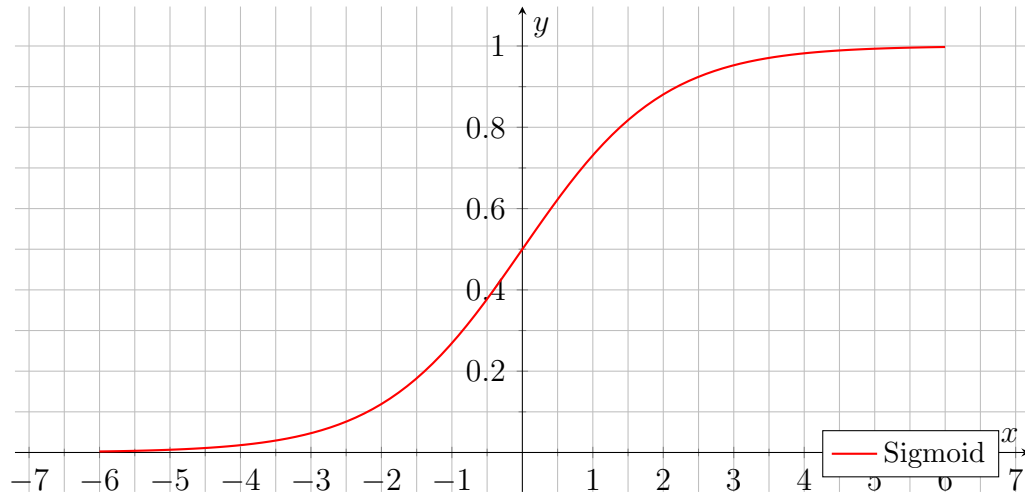


Figure 1.8: Sigmoid Activation Function

1.6 Advanced Neural Network Architectures for Image Compression

1.6.1 Generative Adversarial Network

The popularity of GANs has continued to increase because they have many addressable applications. As Shown in figure 1.9 a GAN is composed of two separate networks: the generator and the discriminator[3]. During the training process, the generator learns how to generate data that mimics real data, whereas the discriminator learns how to differentiate real data from fake data produced by the generator. The training is operated jointly so that both are competing and playing an adversarial zero-sum game. During the training, the generator is trying continuously to generate fake data that deceives the discriminator so that it classifies them as real. However, the discriminator is trying continuously to detect the fake data and not to classify them as real. To solve the adversarial zero-sum game during the training, game theory theorems are used.

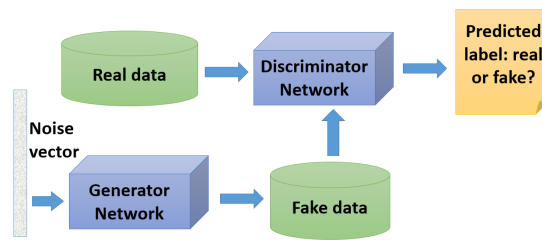


Figure 1.9: Standard architecture of the GAN

1.6.2 Autoencoder architecture

Autoencoders are a specialized class of algorithms that can learn efficient representations of input data with no need for labels. It is a class of artificial neural networks designed for unsupervised learning. Learning to compress and effectively represent input data without specific labels is the essential principle of an automatic decoder. This is accomplished using a two-fold structure that consists of an encoder and a decoder network.

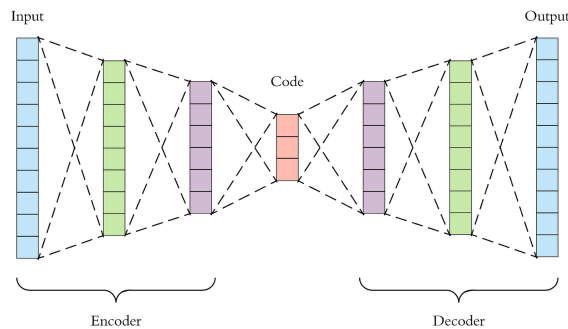


Figure 1.10: Architecture of an Autoencoder

The encoder compresses the input data through its hidden layers, progressively reducing dimensionality and capturing essential features and patterns. These layers culminate in the bottleneck layer, also known as the latent space, where the dimensionality is significantly decreased, representing the compressed encoding of the input data as shown in figure 1.10.

The decoder is like the reverse engineer in the autoencoder system. It takes the compressed version of the input data created by the encoder and tries to rebuild the original input from it. It starts by expanding the compressed data in the bottleneck

layer back to its original size. Then, through a set of hidden layers, it gradually adds more details and complexity to the data, aiming to recreate the essential features and patterns of the original input. Finally, the decoder produces the reconstructed output, ideally making it look as close as possible to the initial input data. So, it's like the decoder's job is to undo what the encoder did, putting the pieces back together to get back to the original picture. The compression relies on the number of convolutional layers, kernel size, stride, and activation functions, and number of channels. Careful selection of network parameters is crucial to balance compression ratio and the quality of reconstructed images. Therefore For a given layer, the output size can be determined by the equation (2.1).

$$\text{Output Size} = \left\lfloor \frac{\text{Input Size} - \text{Kernel Size} + 2 \times \text{Padding}}{\text{Stride}} \right\rfloor + 1 \quad (1.10)$$

By training the network to minimize the difference between the input data and the reconstructed data, the autoencoder can learn to capture the most important features of the input data and ignore any irrelevant or noisy information.[21]

1.6.3 Types of Autoencoders

This section dives into the various kinds of autoencoders, each with its own purpose and strengths.[14]

1.6.3.1 Undercomplete Autoencoder

This is the simplest version of an autoencoder. In this case, we don't have an explicit regularization mechanism, but we ensure that the size of the bottleneck is always lower than the original input size to avoid overfitting. This type of configuration is typically used as a dimensionality reduction technique (more powerful than PCA since its also able to capture non-linearities in the data).

1.6.3.2 Sparse Autoencoder

A Sparse Autoencoder is quite similar to an Undercomplete Autoencoder, but their main difference lies in how regularization is applied. In fact, with Sparse Autoencoders, we don't necessarily have to reduce the dimensions of the bottleneck, but we use a loss function that tries to penalize the model from using all its neurons in the different hidden layers. This penalty is commonly referred to as a sparsity

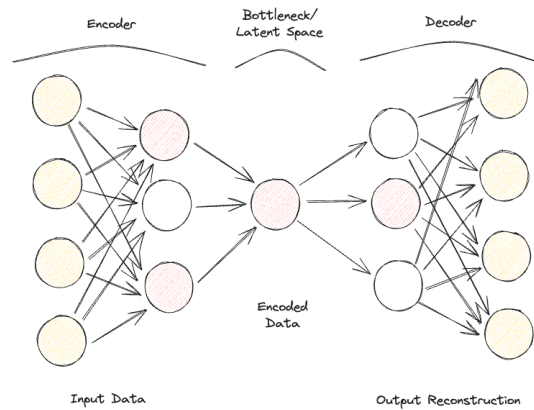


Figure 1.11: Architecture of a Sparse Autoencoder

function, and it's quite different from traditional regularization techniques since it doesn't focus on penalizing the size of the weights but the number of nodes activated. In this way, different nodes could specialize for different input types and be activated/deactivated depending on the specifics of the input data. This sparsity constraint can be induced by using L1 Regularization and KL divergence, effectively preventing the model from overfitting.

1.6.3.3 Contractive Autoencoder

The main idea behind Contractive Autoencoders is that given some similar inputs, their compressed representation should be quite similar (neighborhoods of inputs should be contracted in small neighborhood of outputs). In mathematical terms, this can be enforced by keeping input hidden layer activations derivatives small when fed similar inputs.

1.6.3.4 Denoising Autoencoder

With Denoising Autoencoders, the input and output of the model are no longer the same. For example, the model could be fed some low-resolution corrupted images and work for the output to improve the quality of the images. In order to assess the performance of the model and improve it over time, we would then need to have some form of labeled clean image to compare with the model prediction.

1.6.3.5 Convolutional Autoencoder

To work with image data, Convolutional Autoencoders replace traditional feed-forward neural networks with Convolutional Neural Networks for both the encoder and decoder steps. Updating type of loss function, etc., this type of Autoencoder can also be made, for example, Sparse or Denoising, depending on your use case requirements.

1.6.3.6 Variational Autoencoder

In every type of Autoencoder considered so far, the encoder outputs a single value for each dimension involved. With Variational Autoencoders (VAE), we make this process instead probabilistic, creating a probability distribution for each dimension. The decoder can then sample a value from each distribution describing the different dimensions and construct the input vector, which it can then be used to reconstruct the original input data. One of the main applications of Variational Autoencoders is for generative tasks. In fact, sampling the latent model from distributions can enable the decoder to create new forms of outputs that were previously not possible using a deterministic approach.

1.6.4 Applications of autoencoders

In this section we explore the wide-ranging applications of autoencoders across various domains[2].

1.6.4.1 Dimensionality reduction

Undercomplete autoencoders, used for dimensionality reduction, can serve as a pre-processing step by performing fast and accurate dimensionality reductions without losing much information. Unlike procedures like PCA, which can only perform linear dimensionality reductions, undercomplete autoencoders are capable of large-scale non-linear dimensionality reductions.

1.6.4.2 Image denoising

Autoencoders, such as denoising autoencoders, are highly efficient and accurate for image denoising. Unlike traditional methods that search for noise, autoencoders extract the image from the noisy data by learning its representation, which is then decompressed to form a noise-free image. Consequently, denoising autoencoders can denoise complex images that traditional methods cannot handle.

1.6.4.3 Image and Audio Compression

Autoencoders can compress huge images or audio files while maintaining most of the vital information. An autoencoder is trained to recover the original picture or audio file from a compressed representation.

1.6.4.4 Generation of image and time series data

Variational Autoencoders (VAEs) have the capability to generate both image and time series data. By sampling the parameterized distribution at the bottleneck, discrete values for latent attributes can be generated and passed to the decoder, enabling the generation of image data. Additionally, VAEs excel at modeling time series data like music, showcasing their versatility in various data generation tasks.

1.6.4.5 Recommender System

Using autoencoders, we can use users' preferences to generate personalized suggestions. We can accomplish this by training an autoencoder to learn a compressed representation of the user's history of system interactions and then utilizing this representation to forecast the user's preferences for new items.

1.6.4.6 Anomaly detection

Undercomplete autoencoders can effectively be employed for anomaly detection. When an autoencoder is trained on a specific dataset, it achieves low reconstruction loss for any image sampled from this dataset, accurately reconstructing the image. However, for images not present in the training dataset, the autoencoder fails to reconstruct them effectively due to unfamiliar latent attributes. This results in a high reconstruction loss for outlier images, allowing them to be identified as anomalies when a proper threshold is applied.

1.6.5 Limitations of Autoencoders

Autoencoders present several limitations, one of which is their computational expense, mainly when used in huge data sets or complex models. This significant demand on computational resources can make them costly and inefficient for certain applications. For instance, it tends to overfit thus capturing noises or other spurious things present in training data instead of the underlying data patterns. This overfitting can reduce the model's generalizability and effectiveness when applied to new, unseen data.

1.7 Metrics for Evaluating the Image Compression Quality

1.7.1 Mean Squared Error (MSE)

As explained in Section [1], the Mean Squared Error (MSE) is utilized as a loss function during training. Additionally, MSE can be used as an evaluation metric to assess the quality of image compression and decompression. A lower MSE indicates better compression quality, as it means the decompressed image is closer to the original.

1.7.2 Peak Signal-to-Noise Ratio PSNR

PSNR is a signal processing measurement that compares a given received or processed signal to its original source signal. This comparison allows us to quantify how much a processed signal is faithful to the original, also allowing us to identify possible noises or distortions to the signal. We can say that the PSNR represents a direct relationship of a signal before and after a degradation process. Mathematically, the PSNR is described by the Equations 1.11.

$$\text{PSNR} = 20 \log_{10} \left(\frac{\text{MAX}^2}{\sqrt{\text{MSE}}} \right) \quad (1.11)$$

where MSE is explained in section [1] and MAX is the highest possible value of the signal. In the case of a gray scale image of 8 bits, $\text{MAX} = 255$. The PSNR is inversely proportional to the MSE (Mean Squared Error). The final value of the PSNR is given in decibel.

1.7.3 Structure Similarity Index Method (SSIM)

The Structural Similarity Index (SSIM), is similarity measure between the two images that delivers a value greater than -1 and 1. "The value of +1 denotes that

the images are very similar or coinciding while -1 indicates a different image.” This value typically is scaled to $[0, 1]$ range, which preserve the full meaning in the extremes[47].

SSIM is an error model that is based on perception in that it evaluates the visible errors thereby becoming more human vision based in this process. As the calculations for SSIM are complex, they are explained in detail on this website [10].

1.7.4 Compression Ratio

In the context of image compression, compression specifies the process of minimising the amount of data required to represent an image. The compression ratio, a key metric in this process, is defined as the ratio of the original image file size to the compressed image file size, expressed mathematically by the Equations 1.12.

$$CR = \frac{\text{Size of Original Image}}{\text{Size of Compressed Image}} \quad (1.12)$$

This ratio directly influences the quality of the decompressed image: a higher compression ratio means greater data reduction and typically results in lower image quality due to increased loss of information. Thus, the problem of image compression limits itself to finding an optimal balance between minimizing file size and providing acceptable quality.

1.8 Evaluation metrics for Classification

- **Accuracy:** The ratio of correctly predicted instances to the total instances, measuring overall correctness in classification tasks. It provides a straightforward metric to evaluate model performance.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (1.13)$$

Where:

True Positive (TP): represents correctly predicted positive cases.

True Negative (TN): represents correctly predicted negative cases.

False Positive (FP): represents incorrectly predicted positive cases.

False Negative (FN): represents incorrectly predicted negative cases.

- **F1:** The harmonic mean of precision and recall, providing a single metric to balance both precision and recall. It is especially useful when dealing with imbalanced datasets.

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (1.14)$$

- **Recall:** The ratio of correctly predicted positive instances to all actual positives, indicating the model's ability to identify all relevant instances. High recall is crucial for minimizing false negatives.

$$\text{Recall} = \frac{TP}{TP + FN} \quad (1.15)$$

- **Precision:** The ratio of correctly predicted positive instances to all predicted positives, showing the accuracy of positive predictions. High precision is important for reducing false positives.

$$\text{Precision} = \frac{TP}{TP + FP} \quad (1.16)$$

- **Confusion Matrix:** A table that shows the number of true positive, false positive, true negative, and false negative predictions, summarizing the performance of a classification algorithm. It helps to visualize the model's performance and identify types of errors (see Table 1.1).

Table 1.1: Confusion Matrix

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

1.9 Quantization

Quantization is a technique used to reduce the precision of numerical representations [17], typically converting floating-point numbers to lower bitwidths. This reduction improves storage efficiency and computational speed.

- **Linear Quantization:** Quantizing a float value by multiplying it with a numeric constant (scale factor).
- **Range-Based Quantization:** Calculating the scale factor using the tensor's value range. This can involve using min/max values or a narrower range to exclude outliers. This contrasts with clipping-based methods that apply a fixed or learned clipping function to the tensors.

1.10 Conclusion

Chapter 1 provides distinctive insights into image compression. We discussed the important applications of image compression. Furthermore, we examined how artificial intelligence (AI) enhances these processes. In addition, we focused on deep learning, especially on key concepts such as convolutional neural networks, which underlie many modern AI systems. A significant part of the chapter is devoted to autoencoders, describing their types in detail. Finally, we examined metrics for image compression, including mean squared errors and PSNR. This comprehensive

overview lays the groundwork for understanding the theoretical aspects of image compression and its advancements.

.

Chapter 2

Methodology

2.1 Introduction

This chapter provides the details of our methodology in conducting the project. First, it discusses the dataset used for training, validation, and testing the models. After that, it presents an overview of the architecture of the developed convolutional autoencoder models, the training process, and the methods used for evaluating the compression, decompression and classification tasks.

2.2 Implementation Tools

2.2.1 Software and Platforms

- **Python:** Python is a definable, high-level, object-oriented programming language with dynamic typing and clear, readable syntax, increasing readability and reducing maintenance costs supports modularity and code reuse through modules and packages on. Python’s extensive standards library and cross-platform availability make it incredibly versatile [19]. Its power in AI comes from powerful libraries like TensorFlow, Keras, and PyTorch, which facilitate advanced machine learning and deep learning development. Python’s simplic-

ity, powerful libraries, and large communities make it the top choice for AI and data science applications.

- **Jupyter Notebook:** Jupyter Notebook is a free, open-source web application created by the Jupyter Project. It serves as an interactive laboratory notebook that integrates notes, data, and executable code, allowing users to manipulate data within the notebook itself. Unlike traditional applications like Matlab and Mathematica, Jupyter Notebook is browser-based and supports multiple programming languages, including Python, Julia, and R. This versatility and ease of use make it a popular choice for data science, machine learning, and scientific computing[43].
- **Google Colab:** Google Colab is a cloud-based service built on Project Jupyter that hosts Jupyter notebooks without requiring any local software installation. It supports only Python and stores notebooks in Google Drive, enabling easy sharing and autosaving. Colab provides free access to hardware accelerators such as GPUs and TPUs, making it a powerful tool for machine learning and data analysis. However, Colab does not support simultaneous editing, necessitating serial collaboration. While Colab is free, it has limitations on resource availability and usage, with certain types of code, like media serving and crypto mining, being prohibited. For more reliable resources, Google offers a pro version.[43].

2.2.2 Libraries

- **Pytorch:** PyTorch is a GPU-accelerated tensor computational framework with a Python front end, offering flexibility and speed for deep learning. It integrates well with libraries like NumPy and SciPy, and supports automatic differentiation, neural network layers, optimizers, data loading utilities, and multi-GPU/node setups. PyTorch executes functions immediately, enhancing ease of use and debugging.

- **torchvision:** TorchVision is a PyTorch library for image and video processing, offering essential datasets, model architectures, and transformations for computer vision projects. It includes packages for datasets, IO, models, feature extraction, ops, transforms, and utils. Popular datasets can be loaded in parallel using `torch.utils.data.DataLoader`.
- **Numpy:** NumPy is a fundamental Python library for scientific computing, providing a multidimensional array object and various derived objects. It offers routines for fast array operations such as mathematical computations, shape manipulation, sorting, I/O, and basic linear algebra. NumPy also supports discrete Fourier transforms, statistical operations, and random simulations.
- **Matplotlib:** Matplotlib is an open-source Python library for high-quality visualizations, originally developed by John Hunter in 2002 to visualize brain signals in stroke patients. It replicates MATLAB's graphical capabilities in Python, producing various plots with minimal code. Matplotlib is widely used by Python and NumPy users, and its APIs facilitate easy integration of charts in graphical interface applications.
- **Pillow:** Pillow is a Python Imaging Library that supports opening, manipulating, and saving many image file formats. It provides extensive file format support, an efficient internal representation, and powerful image processing capabilities. Pillow is widely used for image processing tasks in Python.

2.3 Data Collection

Our objective was to train an autoencoder model to compress general, diverse images. Initially, we collected random images from the web, encompassing a variety of subjects such as cities, urban landscapes, nature, humans, and animals. This initial dataset consisted of 540 image samples.

However, as training progressed, we observed that the dataset was relatively

small and lacked sufficient features for the autoencoder to effectively learn from, even after applying data augmentation techniques. To fix this, we added the DIV2K dataset, which has high-quality and diverse images. By mixing our original set with the DIV2K set, we made a custom dataset that provided a more robust and varied set of images for training the autoencoder.

2.3.1 Overview of DIV2K Dataset

The DIV2K dataset has many different types of pictures, including images of people, handmade items, and various places such as cities and villages. It also includes photos of plants, animals, and natural settings like underwater scenes and areas with little light. There are 1,000 pictures in total, and they are all in high 2K resolution. This wide range of content makes DIV2K perfect for training models that need to recognize many different visual details. The high resolution of the images also means they capture small details well, which is important for tasks like image compression and enhancement[5].

2.4 Data Preparation

2.4.1 Image Loading and Preprocessing

To get our dataset ready for training the convolutional autoencoder, we did some steps to clean and organize the data. We changed the size of all pictures to be 256x256 pixels, so they are the same size. We converted the image into PyTorch tensors, which is important for the model to learn from the data.

2.4.2 Data Augmentation

We applied various data augmentation techniques to enhance the diversity of our training dataset and improve model performance. These techniques included

random cropping to take different shots from the images, horizontal and vertical flipping, and slight positional shifts.

2.4.3 Data Splitting

We split the dataset into three subsets : 70% for training, 15% for validation, and 15% for testing. This specific split ratio ensures that the model has sufficient data to learn from while still having sufficient data to validate and test its performance accurately.

2.5 Custom Model Development

In this research, we adopted an autoencoder model for image compression. The choice of Autoencoder was due to its high efficiency in the field of image processing and image compression in particular [37].

In this section, we will build, train, and evaluate the performance of the networks, The goal is to achieve a compression ratio of 17:1, which means that the compressed representation of an image is 5.88% the size of the original image, while ensuring that the reconstructed image is as close as possible to the original.

2.5.1 Experimental Network Designs

During the development of the model, we explored several network architecture to achieve best performance. Initially, We started with a basic autoencoder architecture with two convolutional layers followed by max-pooling in the encoder and two convolutional layers in the decoder. The model's performance was limited in capturing the complex features of the data. we then experimented with more complex and deeper networks consists of four convolutional layers in the encoder, and we mirrored this structure using deconvolutional layers and upsampling operations in the decoder. Then experimented with different stride values (2 and 3) and kernel

sizes (3,5, 6, 7, and 9). Following this, we investigated the effect activation functions and pointwise convolution, to further enhance the model's performance.

2.5.2 Best Performing network architecture

The model consists of an encoder-decoder architecture, in the encoder section of the network, where compression occurs, the input RGB images, sized $3 \times 256 \times 256$, are passed through a series of 2D convolutional layers. Starting with the first convolutional layer employing 46 filters of size 6×6 with a stride of 2 and padding of 2. This results in a feature map of dimensions $46 \times 128 \times 128$. A pointwise convolution [1] is applied to the output through the second convolutional layer with 46 filters of size 1×1 each. A third convolution layer with 46 filters of size 6×6 with a stride of 2 and padding of 2 is applied to the output, this produces a feature map with dimensions of $46 \times 64 \times 64$. Once again we apply a pointwise convolution to the with 46 filters of size 1×1 each. For the fifth layer, another convolutional layer with 76 filters of size 6×6 , a stride of 2, and padding of 2 is applied to the output, resulting in a feature map of dimensions $76 \times 32 \times 32$.

Subsequently, a pointwise convolution with 76 filters of size 1×1 is applied. In the seventh layer, a convolutional layer with 46 filters of size 3×3 , a stride of 2, and padding of 2 is applied, yielding a feature map of dimensions $46 \times 16 \times 16$ as seen in figure 2.1. At the end of the compression phase, we end up with a tensor of size $46 \times 16 \times 16 \times 4$, where the tensor data type is float32. We then used Range-Based quantization [17] to represent the tensor using uint8 values, we obtain a tensor of size $46 \times 16 \times 16$, resulting in a compression ratio of exactly 17:1. The method does not lead to any information loss as it is just enough to accurately represent this value using 8 bits.

In the decoder section of the network, we use a series of 2D transposed convolution layers to reconstruct the original input from the compressed representation obtained in the encoder. The first deconvolutional layer employs 76 filters of size

6x6, with a stride of 2 and padding of 2. This operation results in a feature map with dimensions of 76x32x32. Following this, a second deconvolutional layer with 46 filters of size 6x6, a stride of 2, and padding of 2 is applied to the output, producing a feature map with dimensions of 46x64x64. a third deconvolutional layer with the same parameters was applied resulting in an expanded feature map with dimensions reaching 46x128x128. Moving to the last layer, another deconvolutional layer was applied, this time with 3 filters of size 3x3, a stride of 2, padding of 1, and outputPadding of 1 resulting in a feature map with dimensions of 3x256x256 which matches the size of the input image.

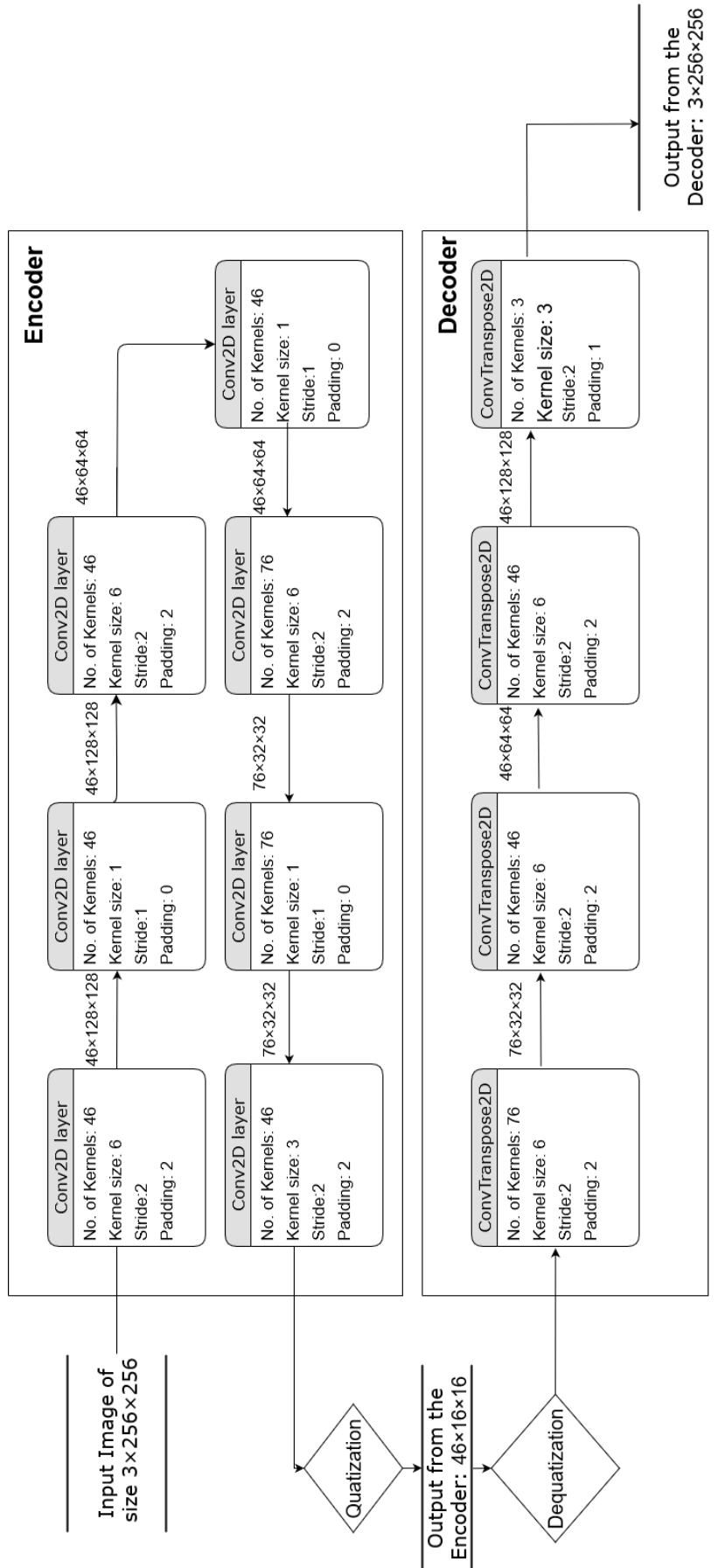


Figure 2.1: Architecture of the Autoencoder

2.5.3 Training the Autoencoder

The training process begins by loading images with a batch size of 32, which balances the need for sufficient gradient updates per epoch while maintaining manageable memory usage. These images are then made into a standard size of 256x256 pixels, which ensures uniform input size, facilitating consistent training and allowing the model to focus on relevant features. The images are subsequently converted into PyTorch tensors for efficient processing. To prevent overfitting, the training data is shuffled before each epoch, ensuring the model sees the data in different orders.

In training, we use a Mean Squared Error (MSE) loss function along with the Adam optimizer, starting with a learning rate of 0.001. We also use a tool called ReduceLROnPlateau [16] that changes the learning rate automatically. It lowers the learning rate when the model's performance stops getting better, making it smaller gradually until it hits a very low number, 0.00001. This dynamic adjustment helps in fine-tuning the learning rate to prevent overshooting the optimal point and to ensure steady convergence.

The training continues for a maximum of 100 epochs. Early stopping is implemented to halt the training process if the testing PSNR (Peak Signal-to-Noise Ratio) fails to improve for 30 consecutive epochs.

2.5.3.1 Fine-Tuning with Layer-Progressive Training

As explained in section 1.6.2, the compression ratio is directly related to the number of layers in the network in hope to find better results, the Fine-tuning technique was employed. The idea was to train the model for a smaller compression ratio, pushing it to its best performance with controlled and focused training, and then adding more layers to the trained model, and training one compression layer and one decompression layer at a time while freezing the parameters of the other layers. This approach ensures that the newly added layers are optimized without disrupting the performance of the previously trained layers.

2.5.3.2 Model evaluation

In order to test our model’s performance, we employed different metrics like Mean Squared Error (MSE) loss and Peak Signal-to-Noise Ratio (PSNR) according to a specific compression ratio. To further validate the quality of the compressed-decompressed images, we used a classification model for tonsil disease detection that was already trained and tested on its original dataset. We retrained this classification model on the dataset that was compressed and decompressed with our compression model. The testing accuracy of the compressed-decompressed dataset was quite similar to that of the original dataset, indicating that no significant loss of information occurred during our compression-decompression process.

2.6 Utilizing Compressed Data for Classification

2.6.1 Overview

Using compressed representations for classification considerably decreases the execution time, which is especially useful when hardware resources are limited. At first, we tried to train a classification model to classify the compressed data. However, the results were underfitted since the autoencoder model’s latent representation is suitable for decompression, not classification-specific characteristics. To solve this issue, an end-to-end compression-classification system was created, the model is designed to perform both classification and compression tasks within a single integrated framework remarkably this technique produced promising results without underfitting, indicating its efficacy in using compressed representations for classification problems.

2.6.2 Data Collection

To prepare the data for a model that does both compression and classification with the compressed data, we have collected 5 Classes of data: Cars, Flowers, Food, Human faces and Sea Animals. In order to manage training and testing time, we have selected 1500 samples randomly from each class. This selection was made to make sure that the process stays practical within a reasonable time frame, considering the extensive computational resources required for large datasets. The datasets used were:

- Cars [33]: The Stanford Cars Dataset contains 16,185 images of 196 classes of cars, making it ideal for fine-grained classification tasks. The dataset includes a comprehensive range of car models and their respective annotations .
- Flowers [34]: This dataset consists of images represent various types of flowers, including roses, sunflowers, tulips, and daisies. It covers a wide range of colors, shapes, and sizes typical of floral species
- Sea Animals [31]: The sea animals dataset includes images of marine creatures such as fish, dolphins, sharks, turtles, and jellyfish. It captures the diverse marine life found in oceans and seas worldwide.
- Human Faces [26]: This dataset contains images of human faces captured from different angles, expressions, and ethnicities. It covers a wide range of facial features and emotions.
- Food [12] : This dataset includes images of various food items, primarily focusing on pizza and junk foods such as burgers, fries, and sandwiches.

2.6.3 Data Preparation

- **Image Loading and Preprocessing:** For image loading and data preprocessing, we followed the same steps as those used in training the autoencoder.

Initially, we resized all images to dimensions of 256 x 256 pixels. then, we converted these images into PyTorch tensors, a crucial step for facilitating model training.

- **Data augmentation:** We employed various techniques to augment the dataset and enrich the training process. These techniques were essential in expanding the dataset size and enhancing the model's ability to generalize effectively. Random transformations such as rotations, horizontal and vertical flipping, and slight positional shifts were applied to introduce variability into the training samples, ensuring that the model learns to recognize objects from different perspectives.
- **Data splitting:** We partitioned the dataset into two subsets: 70% for training, 15% for validation and 15% for testing. This division strategy allows the model to train on a significant portion of the data while reserving a separate portion for evaluating its performance accurately.

2.6.4 Training a Classification Model

2.6.4.1 Model Architecture

The classification network is designed to take the compressed image representation produced by the encoder, with dimensions 46x16x16, and classify it into predefined classes. It begins with convolutional layer employing 64 filters of size 5x5 and padding of 2, followed by a batch normalization layer for improved training stability, and a ReLU activation function as seen in figure 2.2.

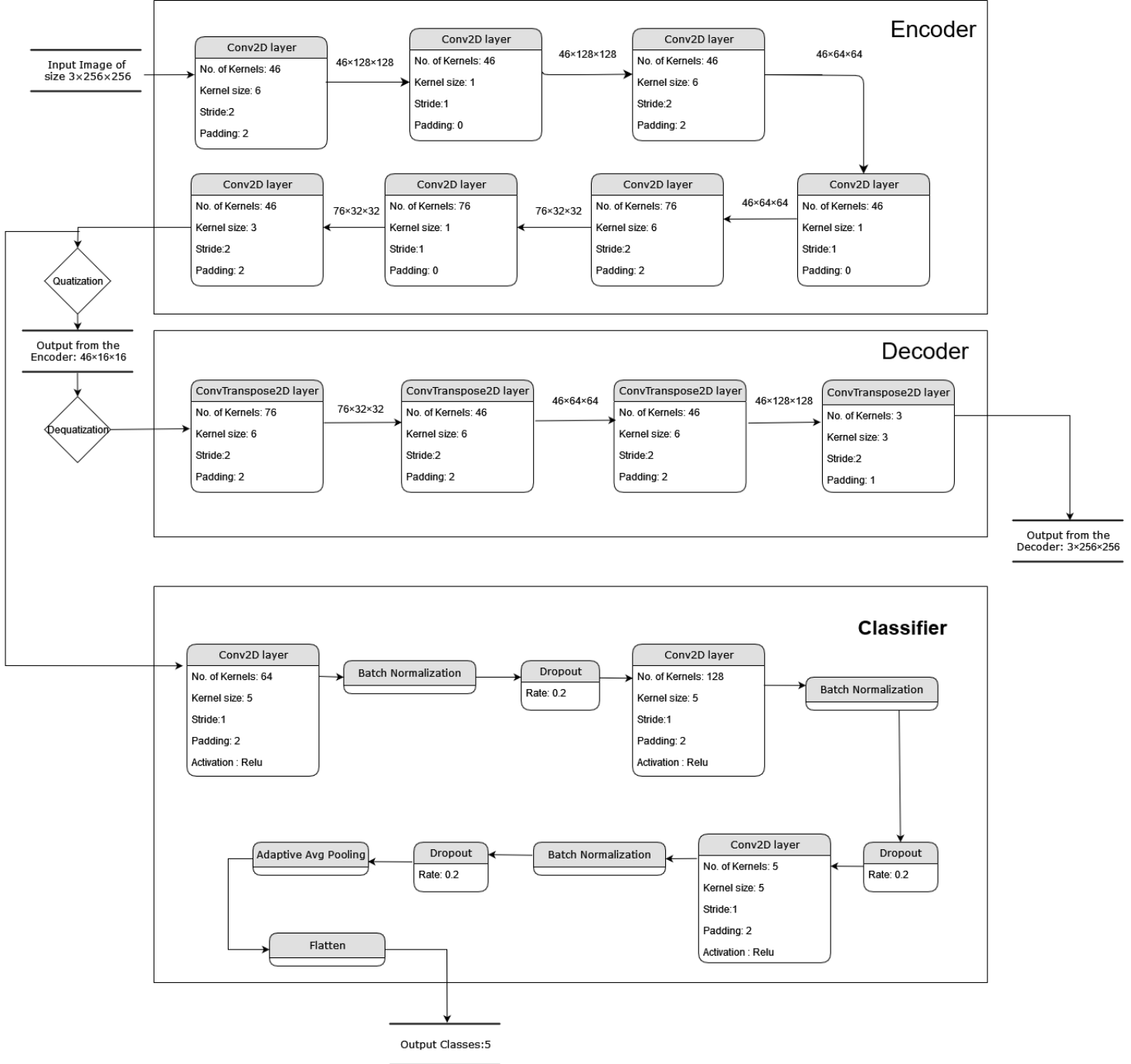


Figure 2.2: Architecture Diagram of the Classification Neural Network

The next convolutional layer has 128 filters of size 5x5, also with a batch normalization layer, a ReLU activation and dropout layer with same padding.

Another convolutional layer with 5 filters of size 5x5 with padding of 2, followed by a batch normalization layer, a ReLU activation function, dropout layer, then an Adaptive Average Pooling layer and a Flattening are applied to the feature map to transform it into a suitable format for the classification task.

2.6.4.2 Training Process

The training process is quite similar to training the autoencoder of the First section, which starts by loading images with a batch size of 32 and to ensure a uniform input size, all the images are resized to 256 x 256 since some are less than this dimension and some are greater. The images are subsequently converted into PyTorch tensors for efficient processing. To promote unbiased learning and prevent overfitting, the training data is shuffled before each epoch, ensuring the model sees the data in different orders.

Each epoch of training involves iterating over the training dataset and performing the following steps for each batch of images and corresponding labels:

- **Forward Pass:** The images are passed through the model to obtain reconstructed images and class logits.
- **Loss Calculation:** The MSE loss is calculated between the reconstructed images and the original images, while the CE loss is computed between the class logits and the actual labels.
- **Backward Pass:** The combined loss ($\text{MSE} + \text{CE}$) is used to compute the gradients, which are then backpropagated through the network.
- **Optimization Step:** The optimizer updates the model parameters based on the computed gradients.

When we test the model, we follow the same steps as during training, but the model is set to "evaluation mode." This means it doesn't use features like dropout and batch normalization that are only for training.

2.6.4.3 Evaluation of Classification and Reconstruction Performances

Performance metrics, including MSE loss, PSNR reconstruction, and CE loss, accuracy, precision, recall, F1 score and confusion matrices are tracked and reported for each epoch. This ensures the model is effectively learning both the reconstruction and classification tasks. Also the classification performance on original versus compressed images is compared to evaluate the impact of the compression on classification accuracy.

2.7 Conclusion

In this chapter, we have provided an overview of the data, including details on data preparation, image loading and preprocessing, and data augmentation. We illustrated the tools used for training and evaluation of the models. We explained the steps taken throughout the development of the model, how we overcame the challenges we faced, and the reasoning behind our decisions until we achieved the desired results.

Chapter 3

Results and Discussion

3.1 Introduction

In Chapter 3, we present the results and discussion of our CNN model. We begin by presenting the results of the Autoencoder based compression-decompression model, showing Training and Validation accuracy curves and the reconstruction accuracy and visual quality of the decompressed images. Next, we present the results of the end-to-end compression-classification model, showing the accuracy of the classification and the evaluation of the compression performance.

3.2 Performance Evaluation of the Compression- Decompression Model

3.2.1 Prototype Autoencoder Network

As discussed in Section 2.5.1, a basic network was trained and the results obtained are showed in the figures 3.1 and 3.2.

The graph showing the Mean Squared Error (MSE) for both the training and validation phases indicates that the error decreases as the training continues, with the lowest error recorded at 2.164. This means the network is getting better at re-

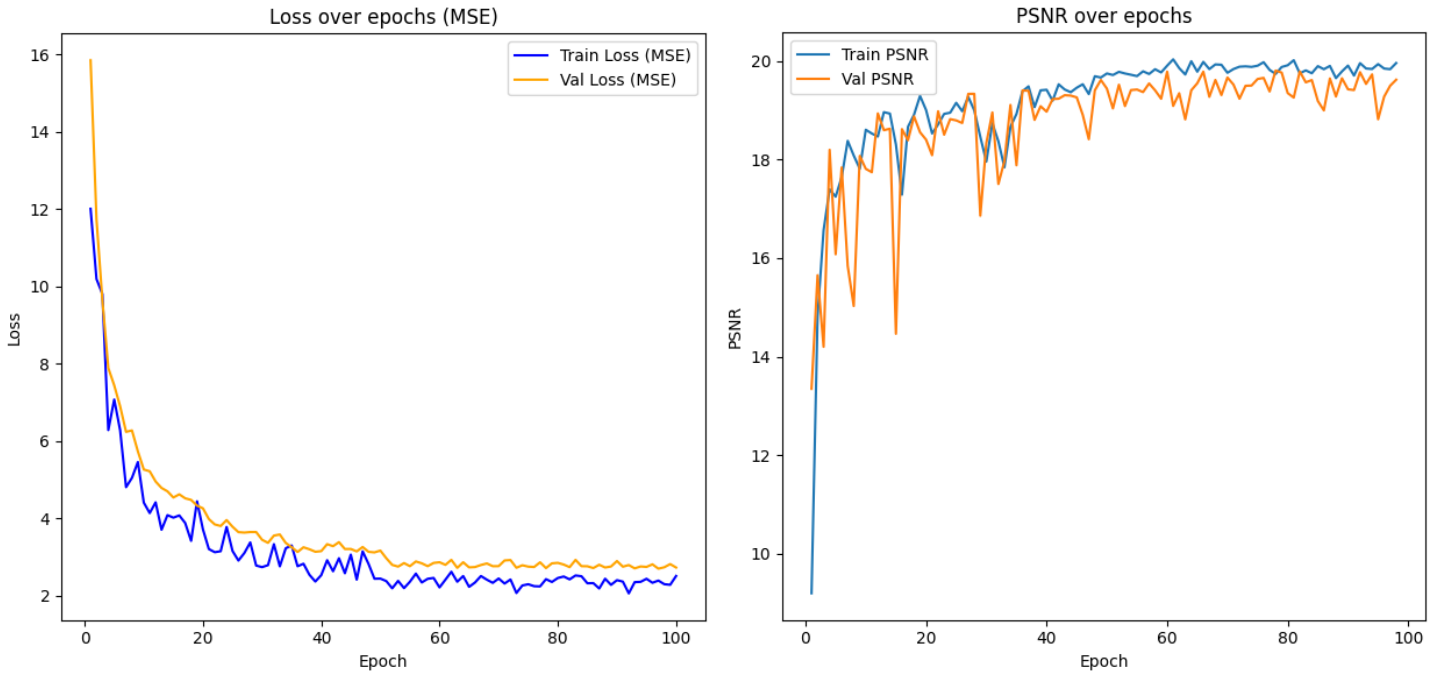


Figure 3.1: MSE loss plotting for the scratch network. Figure 3.2: PSNR plotting for the scratch network.

ducing the error between the original and reconstructed images. However, the graph for Peak Signal-to-Noise Ratio (PSNR) shows only small improvements, starting low and reaching about 20.24 decibels (dB) after 100 training sessions.

The visual assessment of the compressed-decompressed image as shown in figure 3.6 supports these conclusions. The reconstructed images are recognizable but blurry and suffer from loss of fine details.



Figure 3.3: Original Image



Figure 3.4: Original image cropped 256x256



Figure 3.5: Compressed-decompressed from the scratch net

Figure 3.6: Visual Assessment of Scratch Network Performance

3.2.2 Progressive Iterative Network

After several training and testing sessions with various hyperparameters, we found that a network architecture consisting of 4 layers in both the encoder and decoder parts, with a kernel size of 6 and a stride of 2, yields the best results. This is evident in the PSNR and MSE graphs (see Figure 3.7)., However, a visual assessment of the reconstructed image quality indicates that further enhancement is necessary. The reconstructed images exhibit noticeable artifacts, specifically in the form of obvious squares, as illustrated in Figure 3.10. This shows that, while the chosen network architecture works well in terms of PSNR, more work is needed to improve the visual quality of the compressed-decompressed images.

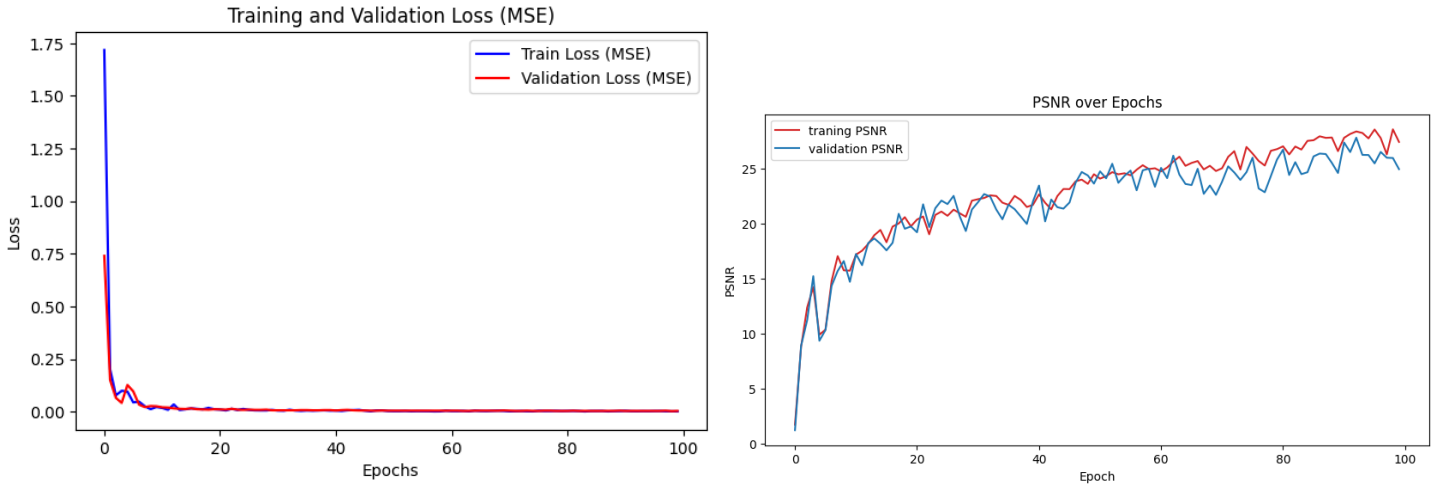


Figure 3.7: PSNR and MSE loss plotting for the refined network



Figure 3.8: Original image



Figure 3.9: Compressed-decompressed

Figure 3.10: Original image vs compressed-decompressed.

3.2.3 Optimized Autoencoder Model: Peak Performance Achieved

Finally, after experimenting with activation functions, Batch Normalization we found that pointwise convolution after each layer in the encoder part resulted in the better performance, as shown in the PSNR and MSE loss graphs 3.11 and 3.12. We

also added more diverse and complex images to the training set, which improved the model's ability to generalize. This led to achieving good PSNR values around 36 dB with a compression ratio of 17:1. In visual assessment, the difference between the original and reconstructed images was almost unnoticeable, as shown in figure 3.15, indicating high-quality reconstruction.

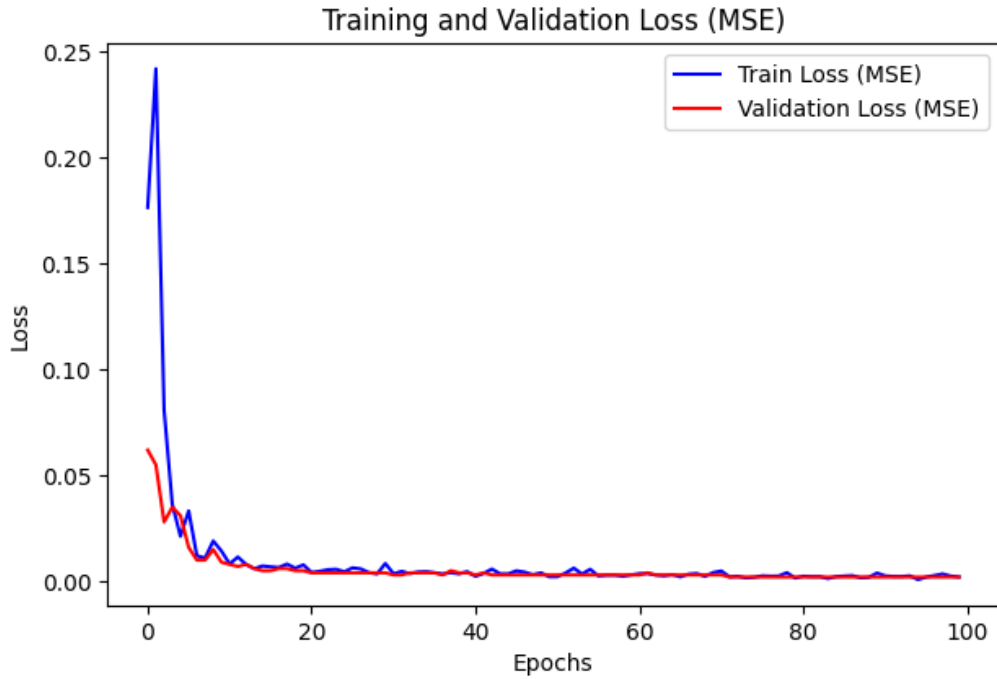


Figure 3.11: MSE loss for the best network.

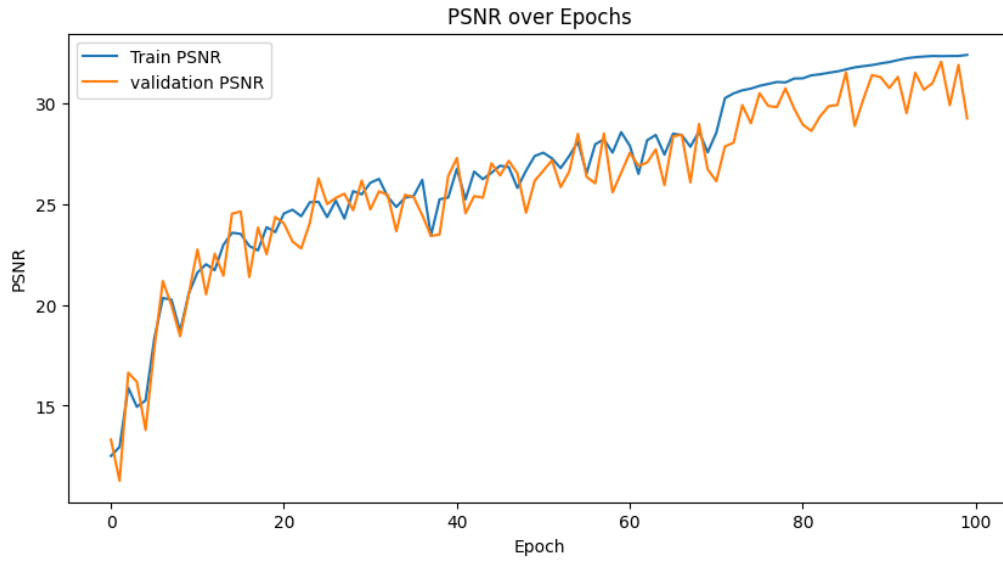


Figure 3.12: PSNR plotting for the optimized network



Figure 3.13: Original image



Figure 3.14: Compressed-decompressed

Figure 3.15: Original image vs compressed-decompressed.

For further evaluation of the quality of the reconstructed images, we assessed the accuracy with a classification model for tonsil disease. We evaluated the model's accuracy using both the original dataset and the dataset compressed-decompressed with our model. The results showed the same accuracy and loss for both cases as seen in the figure 3.16 and 3.17. This indicates that the images did not lose any information that could affect the classification performance, which confirms the high

visual quality of the images after compression and decompression.

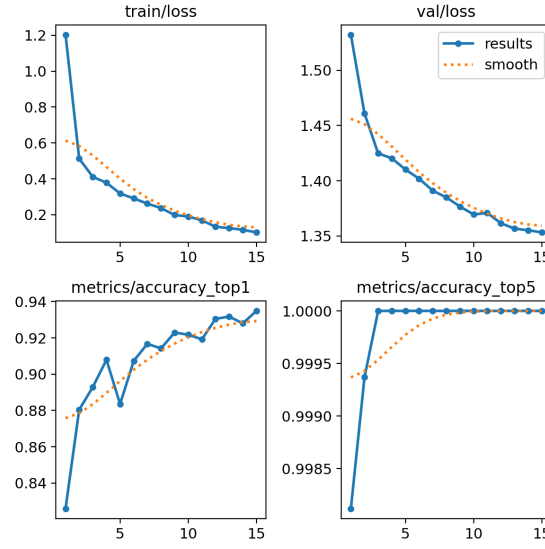


Figure 3.16: Evaluation of the classification with original data

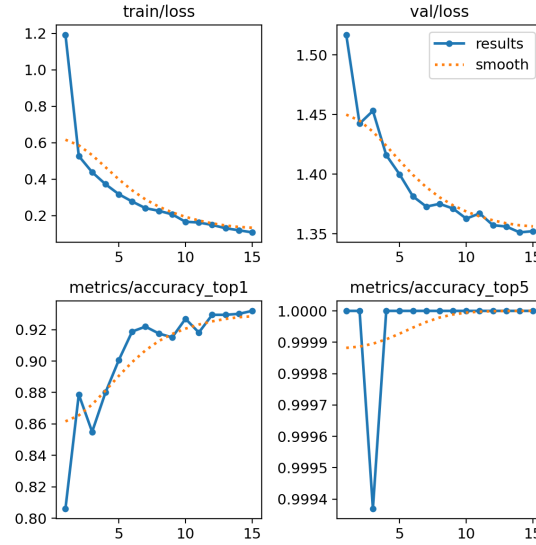


Figure 3.17: Evaluation of the classification with reconstructed data

3.2.4 Fine-Tuning Training Results

As discussed in section 2.5.3.1, Fine-Tuning technique was used to train the model. By training a smaller model then adding more layers, we could easily make

trade-offs between compression ratio and reconstructed image quality. The process consisted of 4 training sessions. In the first two, the model was too small to result in any compression. At the third training session, the compression ratio was 3:1, and achieved really good results as seen in Figure 3.18.

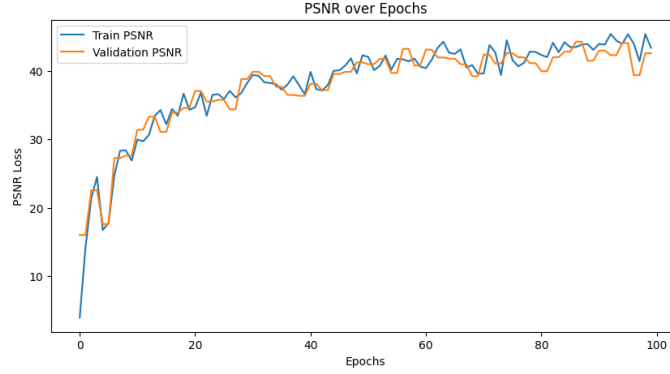


Figure 3.18: PSNR of the train and validation

At the fourth training session, the compression ratio was 17:1, and the model gave the best result so far as seen in Figure 3.19.

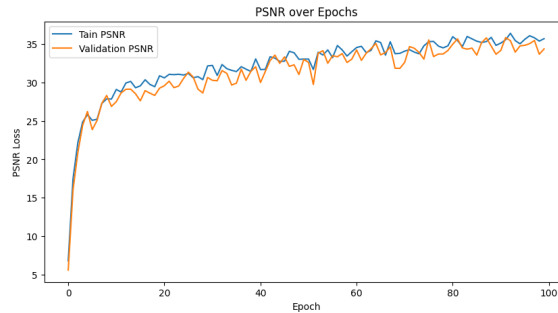


Figure 3.19: PSNR of the train and validation

3.3 Performance Analysis of Classification with Compressed Images

As discussed in Section 2.6.4.1, a network was trained to perform both compression and classification on the compressed data. The following results were obtained: The graphs of MSE and Cross Entropy Loss, shown in Figures 3.20 and 3.21, indicate

good results for both classification and reconstruction. Additionally, the accuracy graph in Figure 3.24 shows satisfactory performance. However, the PSNR values are not as high as those achieved by the autoencoder PSNR graph, likely due to the use of a non-diverse dataset that limits the encoder and decoder networks from learning more features, which would help increase the PSNR values.

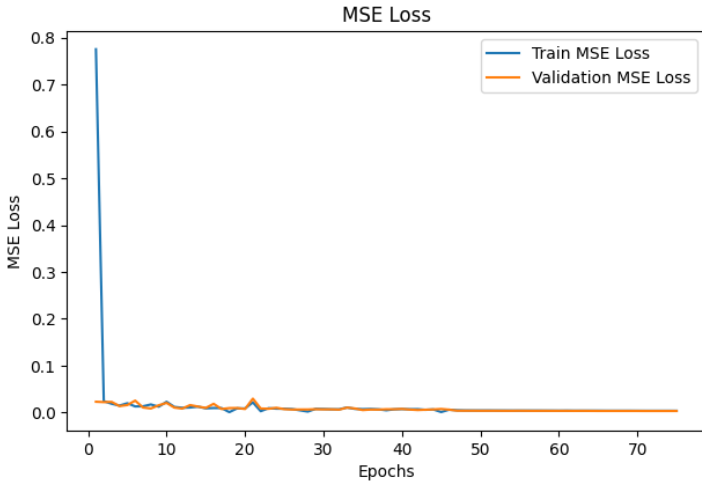


Figure 3.20: MSE Loss

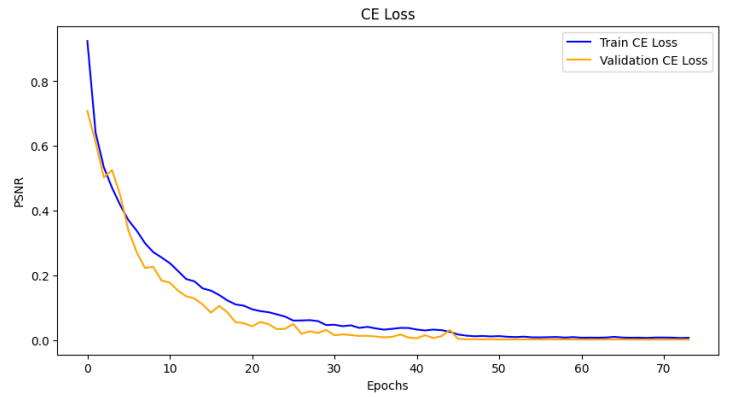


Figure 3.21: Cross entropy Loss

Figure 3.22: The loss plot for both reconstruction and Classification.

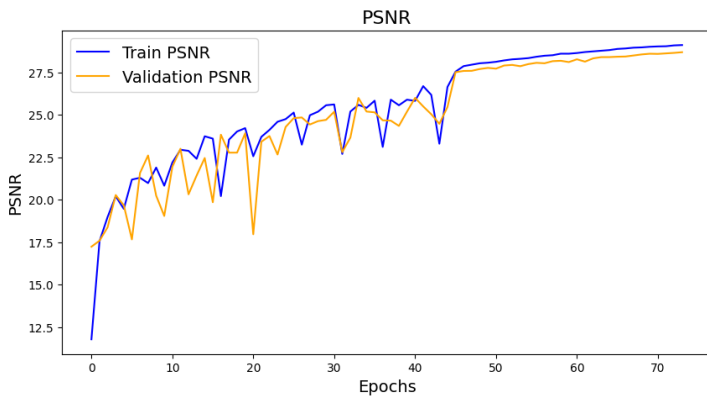


Figure 3.23: PSNR of the train and validation

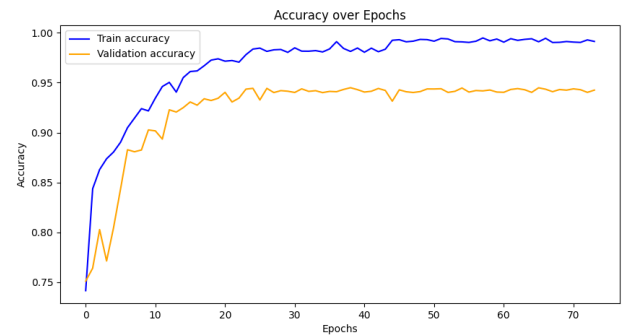


Figure 3.24: Accuracy of train and validation

Figure 3.25: PSNR and accuracy plots for Reconstruction and classification.

Visual assessment of the figures in 3.28 indicates that the quality of the reconstructed image is reasonably high. The reconstructed image retains most of the visual details of the original image, although some minor artifacts may still be

present. This visual confirmation, along with the metrics, supports the effectiveness of the compression and reconstruction process.



Figure 3.26: Original image for classification and reconstruction



Figure 3.27: reconstructed image

Figure 3.28: Original image vs compressed-decompressed.

The classification report, detailed in Table 3.1, presents precision, recall, and F1-score metrics for each class on the test data. The high values across these metrics indicate the model's strong performance in distinguishing between different classes. The precision, recall, and F1-score values for all classes are above 0.90, demonstrating the model's effectiveness in accurately classifying images across various categories.

Table 3.1: Classification report with precision, recall, and F1-score for each class.

Class	Precision	Recall	F1-Score
Cars	0.95	0.97	0.96
Human	0.93	0.90	0.92
Sea	0.96	0.97	0.97
Flowers	0.94	0.96	0.95
Food	0.92	0.91	0.91

The confusion matrix in Figure 3.29 provides a visual representation of the classi-

fication results on an unseen data, showing the counts of true positive, false positive, true negative, and false negative predictions. The matrix indicates that the model performs exceptionally well in correctly classifying most instances, with very few misclassifications. This further validates the model’s high performance metrics observed in the classification report.

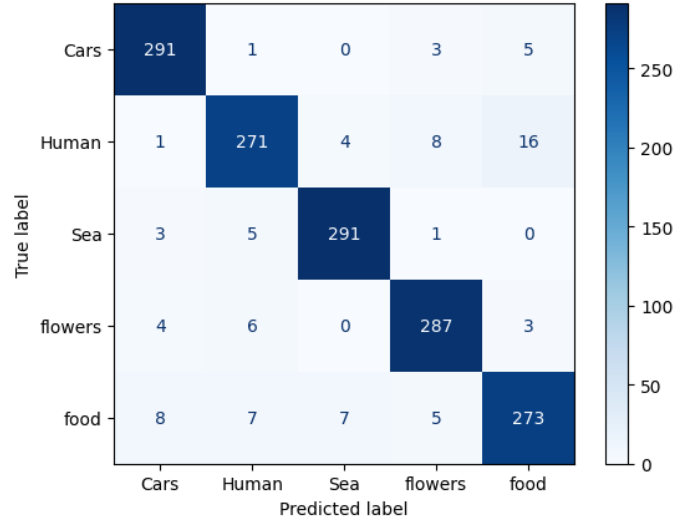


Figure 3.29: Confusion Matrix

3.4 Comparision

In this study, we have compared our Autoencoder-based image compression model with several traditional and modern compression methods using the Kodak dataset. The key metrics used for comparison are Peak Signal-to-Noise Ratio (PSNR) and Compression Ratio, as shown in Table 3.2. This comparison was conducted on the Kodak dataset[29], which serves as unseen data for our model. Our model achieved a PSNR of 32 dB at a compression ratio of 17:1. When we compare this with traditional methods like JPEG and JPEG 2000, we see that our model is outperformed. Specifically, JPEG has a PSNR of 34 dB, and JPEG 2000 has a PSNR of 36.8 dB. These values were derived from [36] (Section 4, Figure 5) by projecting from a graph at a Bits Per Pixel (BPP) of 1.3, which corresponds to a

compression ratio of 17:1. Modern methods like COIN[18] and IPF[50] show varying performance. COIN achieves a PSNR of approximately 31 dB, which is slightly lower than our model, while IPF achieves a higher PSNR of approximately 32.8 dB for a compression ratio of 17:1. These methods use advanced deep learning techniques, which help them perform well. Despite our model’s lower PSNR compared to some of these methods, it shows great promise. Notably, our model surpassed COIN, demonstrating its potential. Convolutional Neural network-based compression is a growing field, and there is significant room for improvement. Our model’s performance is an excellent starting point, and with further development, it could achieve even better results.

Table 3.2: Comparison of PSNR and Compression Ratio between different image compression methods on the Kodak dataset.

Method	PSNR (dB)	Compression Ratio	Dataset
Our model	32	17:1	Kodak
JPEG	34	17:1	Kodak
JPEG 2000	36.8	17:1	Kodak
COIN	31	17:1	Kodak
IPF	32.8	17:1	Kodak

3.5 Conclusion

In this chapter, we introduced our concept and described the results of our study. The model demonstrated significant achievements in compressing photos while maintaining excellent quality. This was validated by the PSNR values and how the photos appeared. These improvements were accomplished by optimizing the model’s hyperparameters and employing transfer learning, demonstrating that the model can effectively learn from data. Furthermore, the classification results utilizing compressed data revealed that the compressed photos retained crucial in-

formation. This means that the model is useful not just for compression, but also for computer vision tasks like classification.

Genenral Conclusion

Since images form a significant part of internet traffic, compressing them without quality loss is essential. This optimizes storage space, bandwidth usage, and improves web application performance.

In this project, we designed an Autoencoder model for image compression. Our model focuses on achieving high compression ratios by learning to encode and decode image data effectively. Additionally, our model addresses the challenge of using compressed data directly in training CNN models.

The results of our project show that our Autoencoder model is very effective at compressing images, achieving an average PSNR value of 32 dB with a compression ratio of 17:1, which is a really good result compared to recent published work. Although the classification of images using compressed data didn't yield the desired outcome, we were able to overcome this by designing an end-to-end compression-classification model, which achieved really high accuracy.

Despite the promising results, our model has certain limitations. while the model performs well on a wide range of images, its effectiveness may decrease with highly complex images. Further research is needed to enhance the model's efficiency. Future directions in this field include developing methods for using compressed data in training machine learning models to fully leverage the benefits of our compression approach. Additionally, it is essential to develop techniques to accelerate the compression process, enabling the model to be utilized for real-time video compression.

Bibliography

- [1] Pointwise convolution. <https://paperswithcode.com/method/pointwise-convolution>.
- [2] Unleashing the power of autoencoders: Applications and use cases. *Analytics Vidhya*, May 2023.
- [3] Data-efficient domain adaptation for semantic segmentation, Year of Publication.
- [4] Abien Fred Agarap. Deep learning using rectified linear units (relu). *arXiv preprint arXiv:1803.08375*, 2018.
- [5] Eirikur Agustsson and Radu Timofte. Ntire 2017 challenge on single image super-resolution: Dataset and study. In *2017 IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 1122–1131, 2017.
- [6] Ahmed Hussein Ali, Mohanad G Yaseen, Mohammad Aljanabi, and Saad Abbas Abed. Transfer learning: A new promising techniques. *Mesopotamian Journal of Big Data*, 2023:29–30, 2023.
- [7] Johannes Ballé, Valero Laparra, and Eero P. Simoncelli. End-to-end optimized image compression. *IEEE Transactions on Image Processing*, 29:490–505, 2018.
- [8] James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13(2):281–305, 2012.

- [9] Olivier Chapelle, Bernhard Scholkopf, and Alexander Zien, editors. *Semi-Supervised Learning*. MIT Press, 2010.
- [10] Wikipedia contributors. Structural similarity, 2024.
- [11] Nan Cui. Applying gradient descent in convolutional neural networks. In *Journal of Physics: Conference Series*, volume 1004, page 012027. IOP Publishing, 2018.
- [12] Zgur Dan. Cv pizza images dataset. Roboflow, <https://universe.roboflow.com/cv-9bj6o/cv-pizza-images-dataset>, 2021.
- [13] Databricks. Artificial neural network. <https://www.databricks.com/glossary/artificial-neural-network>. Accessed: 2024-05-15.
- [14] DataCamp. Introduction to autoencoders. <https://www.datacamp.com/tutorial/introduction-to-autoencoders>, 2024. Accessed: 2024-05-19.
- [15] Sachin Dhawan. A review of image compression and comparison of its algorithms. *International Journal of electronics & Communication technology*, 2(1):22–26, 2011.
- [16] PyTorch Documentation. Reducelronplateau.
- [17] PyTorch Documentation. Quantization, 2024.
- [18] Emilien Dupont, Adam Goliński, Milad Alizadeh, Yee Whye Teh, and Arnaud Doucet. Coin: Compression with implicit neural representations. *arXiv preprint arXiv:2103.03123*, 2021.
- [19] Python Software Foundation. General python essays, 2024. Accessed: 2024-05-19.
- [20] Salvador García, Julián Luengo, and Francisco Herrera. Data preprocessing in data mining. *Data Mining and Knowledge Discovery*, 1(2):25–72, 2015.

- [21] GeeksforGeeks. Auto encoders. <https://www.geeksforgeeks.org/auto-encoders/>, 2024. Accessed: 2024-05-19.
- [22] Glinteco. Benefits of image compression, 2024. Accessed: 2024-05-19.
- [23] Xavier Glorot, Antoine Bordes, and Yoshua Bengio. Deep sparse rectifier neural networks. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, pages 315–323, 2011.
- [24] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [25] Google Developers. Webp image format, 2024.
- [26] Ashwin Gupta. Human faces. Kaggle, <https://www.kaggle.com/ashwingupta3012/human-faces>, 2020.
- [27] Aurélien Géron. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O’Reilly Media, 2nd edition, 2019.
- [28] Geoffrey E. Hinton and Ruslan R. Salakhutdinov. Reducing the dimensionality of data with neural networks. *Science*, 313(5786):504–507, 2006.
- [29] Kodak. Kodak lossless true color image suite. <http://r0k.us/graphics/kodak/>.
- [30] Sotiris B. Kotsiantis, Dimitris Kanellopoulos, and Panayiotis E. Pintelas. Data preprocessing for supervised learning. *International Journal of Computer Science*, 1(2):111–117, 2006.
- [31] Vencer Lanz. Sea animals image dataset. Kaggle, <https://www.kaggle.com/vencerlanz09/sea-animals-image-dataset>, 2021.
- [32] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015.

- [33] Jessica Li. Stanford cars dataset. Kaggle, <https://www.kaggle.com/datasets/jessicali9530/stanford-cars-dataset>, 2023.
- [34] Alx Mamayev. Flowers recognition. Kaggle, <https://www.kaggle.com/alxmamaev/flowers-recognition>, 2018.
- [35] Kevin P. Murphy. Machine learning: A probabilistic perspective. *MIT Press*, 2012.
- [36] Yichen Qian, Ming Lin, Xiuyu Sun, Zhiyu Tan, and Rong Jin. Entroformer: A transformer-based entropy model for learned image compression. *arXiv preprint arXiv:2202.05492*, 2022.
- [37] Mau Quach, Giuseppe Valenzise, and Frederic Dufaux. Deep autoencoder-based lossy geometry compression for point clouds. *arXiv preprint arXiv:1905.03691*, 2019.
- [38] Sebastian Ruder. An overview of gradient descent optimization algorithms. *arXiv preprint arXiv:1609.04747*, 2016.
- [39] Stuart J Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Prentice Hall, 3rd edition, 2010.
- [40] Connor Shorten and Taghi M. Khoshgoftaar. A survey on image data augmentation for deep learning. *Journal of Big Data*, 6(1):1–48, 2019.
- [41] Coursera Staff. Coursera, 2024.
- [42] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.
- [43] TechTarget Contributor. Why and how to use google colab, 2021.
- [44] Gaurav Vijayvargiya, Sanjay Silakari, and Rajeev Pandey. A survey: Various techniques of image compression. *CoRR*, abs/1311.6877, 2013.

- [45] Gregory K. Wallace. The jpeg still picture compression standard. *Communications of the ACM*, 34(4):30–44, 1992.
- [46] Hua Wang, Cuiqin Ma, and Lijuan Zhou. A brief review of machine learning and its application. In *2009 international conference on information engineering and computer science*, pages 1–4. IEEE, 2009.
- [47] Zhou Wang, Alan C Bovik, Hamid R Sheikh, and Eero P Simoncelli. Image quality assessment: from error visibility to structural similarity. *IEEE transactions on image processing*, 13(4):600–612, 2004.
- [48] Zijie J. Wang, Robert Turko, Omar Shaikh, Haekyu Park, Nilaksh Das, Fred Hohman, Minsuk Kahng, and Duen Horng (Polo) Chau. Cnn explainer: Learning convolutional neural networks with interactive visualization. In *IEEE Transactions on Visualization and Computer Graphics (TVCG)*. IEEE, 2021.
- [49] Marcelo J. Weinberger, Gadiel Seroussi, and Guillermo Sapiro. The loco-i lossless image compression algorithm: Principles and standardization into jpeg-ls. *IEEE Transactions on Image Processing*, 9(8):1309–1324, 2000.
- [50] Yunfan Zhang, Ties Van Rozendaal, Johann Brehmer, Markus Nagel, and Taco Cohen. Implicit neural video compression. *arXiv preprint arXiv:2112.11312*, 2021.